MACHINE LEARNING

Neural networks and language models from first principles

UNIT 1

Learning Machines

Tensors, gradients, probability, optimisers, networks, generalisation and representations, built from first principles

1.1 Tensors and what they cost

1.2 Gradients by machine

1.3 Models as probability

1.4 Optimisers

1.5 Neural networks

1.6 Generalisation

1.7 Training deep networks

1.8 Representations

First edition, October 2026

Contents

About this unit . . . 3
Before you start . . . 3
What this unit needs . . . 3
1.1 Tensors and what they cost . . . 4
Core ideas . . . 4
Reading . . . 9
Labs . . . 9
Problem set . . . 10
1.2 Gradients by machine . . . 12
Core ideas . . . 12
Reading . . . 20
Labs . . . 20
Problem set . . . 22
1.3 Models as probability . . . 23
Core ideas . . . 23
Reading . . . 27
Labs . . . 28
Problem set . . . 29
1.4 Optimisers . . . 30
Core ideas . . . 30
Reading . . . 36
Labs . . . 36
Problem set . . . 37
1.5 Neural networks . . . 39
Core ideas . . . 39
Reading . . . 44
Labs . . . 45
Problem set . . . 46
1.6 Generalisation . . . 47
Core ideas . . . 47
Reading . . . 51
Labs . . . 51
Problem set . . . 53
1.7 Training deep networks . . . 54
Core ideas . . . 54
Reading . . . 58
Labs . . . 58
Problem set . . . 60
1.8 Representations . . . 62
Core ideas . . . 62
Reading . . . 69
Labs . . . 69
Problem set . . . 70
Sign-off . . . 72

Machine Learning, First edition, October 2026. © 2026 O'Neill Group LLC. PyTorch is a trademark of the Linux Foundation. Apple, Apple Silicon and Metal are trademarks of Apple Inc. NVIDIA and CUDA are trademarks of NVIDIA Corporation. Vulkan is a registered trademark of the Khronos Group Inc.

Hugging Face is a trademark of Hugging Face, Inc. All other trademarks belong to their owners. The models and datasets the course uses keep their own licences, listed in the Handbook.

UNIT 1

About this unit

Unit 1 builds the machinery that every later model runs on. It starts with tensors and the cost of arithmetic on them, then writes an automatic differentiation engine and grows it into `tiny`, a small tensor library. It derives the standard losses from probability, implements the optimisers in common use, checks each against PyTorch's, and trains neural networks with them. It then studies why models generalise or fail to, why deep networks are hard to train and what makes them trainable, and ends with the representations that networks learn. Chapter 1.5 trains the same network in `tiny` and in PyTorch, and from Chapter 1.7 on, programs use PyTorch directly.

Chapter	You build	Hours
1.1	Views and strides, broadcasting, and a matrix multiplication from loops to the GPU	5
1.2	A scalar autodiff engine and `tiny`, a tensor library checked against PyTorch	7
1.3	Regression three ways, softmax regression on MNIST, a stable softmax	5
1.4	Six optimisers matched to PyTorch's, paths through a valley, a range test	5
1.5	Initialisation measured through depth, and MNIST to 97% in `tiny` and PyTorch	6
1.6	Overfitting reduced four ways, and double descent reproduced	5
1.7	A 30-layer network made trainable, normalisation by hand, three bugs found	6
1.8	A convolutional network on CIFAR-10, embeddings and contrastive learning	7

Before you start

Read the Handbook, and run `uv run -m aif.selftest` from the companion code. The unit assumes the mathematics summarised in the Handbook's maths part: vectors and matrices, derivatives and the chain rule, and the basics of probability.

What this unit needs

Any recent computer. Chapters 1.1 to 1.7 run on the CPU in minutes; Chapter 1.8 trains convolutional networks and is best run on a GPU, although it also runs, much more slowly, on a CPU.

CHAPTER 1.1

Tensors and what they cost

TIME	5 hours
YOU BUILD	A tour of array memory through strides and views; the broadcasting rule written out and checked against NumPy; three ways to compute a table of distances, the fastest three thousand times quicker than the slowest; and a matrix multiplication taken from Python loops to BLAS and the GPU, timed against your machine's peak.
NEEDS	The companion code installed as in the Handbook's setup part. No machine learning yet.

WHY IT MATTERS

Every model in this course, from a linear regression to a language model, is arithmetic on arrays of numbers. How fast it trains, how large it can grow and whether it fits in memory at all depend mostly on how that arithmetic runs on the machine: how arrays are laid out, how many bytes each operation moves, and whether the processor spends its time computing or waiting. This chapter builds the vocabulary for those questions. Later chapters use it throughout, and Unit 4 returns to it in depth.

SKIP TEST

1. A `float32` array has shape `(3, 4)` and strides `(16, 4)`. What are the shape and strides of its transpose, and does taking the transpose copy any data?
2. What shape does broadcasting `(8, 1, 6, 1)` with `(7, 1, 5)` produce, and why does `(3, 4)` with `(3,)` fail?
3. How many floating-point operations does multiplying two $n \times n$ matrices take, and how many bytes must it move at the least?
4. Why can copying an array through its transpose be forty times slower than copying it in memory order?

If all four are easy, skim the chapter and do Lab 1.1.4.

Core ideas

A tensor is a buffer and a recipe

In machine learning, *tensor* means an array with any number of dimensions: a scalar has none, a vector one, a matrix two, a batch of colour images four. Whatever its dimensions, the numbers live in one flat block of memory. What makes the block a tensor is a small recipe kept beside it: a *shape*, the length of each dimension; a *data type*, such as `float32`; and *strides*, the number of bytes to step in memory to move one place along each dimension. Element $(i_0, i_1, \ldots, i_{k-1})$ lives at

$$\text{offset} = \sum_{d=0}^{k-1} i_d s_d$$

bytes from the start of the buffer, where s_d is the stride of dimension d. A 3×4 array of `float32` stored row by row has strides $(16, 4)$: the next column is four bytes on, the next row sixteen. That order, last index fastest, is *row-major* or *C order*, NumPy's and PyTorch's default; such an array is *contiguous*.

Because the recipe and the buffer are separate, many operations change only the recipe. A transpose swaps the strides; a slice moves the start and multiplies a stride; a reshape of a contiguous array computes new strides. None of them copies any data. The results are *views*, new tensors that share the original memory.

Six tensors described by their recipes — code/u1/strides.py

```
a = np.arange(12, dtype=np.float32).reshape(3, 4)
describe("a", a, a)
describe("a.T", a.T, a)
describe("a[:, 1:3]", a[:, 1:3], a)
describe("a[::2]", a[::2], a)
describe("a.reshape(4, 3)", a.reshape(4, 3), a)
describe("a.T.reshape(12)", a.T.reshape(12), a)
```

Five views and one copy

```
$ uv run -m u1.strides
a                 shape (3, 4)   strides (16, 4)   contiguous True   shares a's memory
a.T               shape (4, 3)   strides (4, 16)   contiguous False  shares a's memory
a[:, 1:3]         shape (3, 2)   strides (16, 4)   contiguous False  shares a's memory
a[::2]            shape (2, 4)   strides (32, 4)   contiguous False  shares a's memory
a.reshape(4, 3)   shape (4, 3)   strides (12, 4)   contiguous True   shares a's memory
a.T.reshape(12)   shape (12,)    strides (4,)      contiguous True   a copy
```

The last line is the exception. The transpose is not contiguous, so no choice of strides can read it as a flat run of twelve numbers in its new order, and `reshape` has to copy. NumPy copies silently; PyTorch's `view` refuses and its `reshape` copies. Knowing which operations copy matters for speed and, as the next lab shows, for correctness: writing through a view changes the original.

Strides can also take values that no reshape produces. A stride may be zero, which repeats one element along a dimension, and two dimensions may overlap. `as_strided` builds such views directly, here eight overlapping windows of three over a signal of ten:

Sliding windows without a copy: both dimensions step by one float — code/u1/strides.py

```
signal = np.arange(10, dtype=np.float32)
windows = np.lib.stride_tricks.as_strided(
    signal, shape=(8, 3), strides=(4, 4), writeable=False
)
print("sliding windows of 3 over 0..9, without copying:")
print(windows)
```

Broadcasting, below, uses zero strides internally, and convolution libraries use overlapping windows like these to turn a convolution into a matrix multiplication.

Memory layout and speed

Memory is fast to read in order and slow to read at random. A processor fetches memory in *cache lines* of 64 or 128 bytes and keeps recently used lines in caches close to the arithmetic units. Reading the rest of a line it has fetched costs almost nothing, while reading one float from each of many distant lines costs a full trip to memory for each. The same rule holds on a GPU, as Unit 4 shows. The program copies an 8192×8192 array twice: once in memory order, and once through its transpose, so that the reads walk down columns.

The same copy in two orders — code/u1/strides.py

```
big = np.random.default_rng(0).random((8192, 8192), dtype=np.float32)
straight = median_time(lambda: big.copy(), runs=5)
across = median_time(lambda: big.T.copy(), runs=5)
gb = 2 * big.nbytes / 1e9
print(
    f"copy 256 MiB in memory order: {straight * 1e3:5.1f} ms ({gb / straight:5.1f} GB/s)"
)
print(f"copy it through a transpose:  {across * 1e3:5.1f} ms ({gb / across:5.1f} GB/s)")
```

Copying in memory order and across it on the M2 Pro

```
copy 256 MiB in memory order:   6.1 ms ( 87.9 GB/s)
copy it through a transpose:  238.6 ms (  2.2 GB/s)
```

Both copies read 256 MiB, write 256 MiB and perform the same arithmetic, and one is nearly forty times faster than the other. This is the first example of a theme of the whole course: *an operation's cost is usually set by the data it moves*.

Broadcasting

Arithmetic on two tensors of different shapes is defined by *broadcasting*. Align the shapes at the right, counting a missing dimension as 1. Along each dimension the sizes must be equal, or one of them must be 1, and a dimension of size 1 is stretched, with stride zero, to match the other. The result has the larger size in every dimension.

The broadcasting rule, dimension by dimension from the right — code/u1/broadcast.py

```python
def broadcast_shape(a: tuple, b: tuple) -> tuple:
    out = []
    for x, y in zip_longest(reversed(a), reversed(b), fillvalue=1):
        if x != y and 1 not in (x, y):
            raise ValueError(f"cannot broadcast {a} with {b}")
        out.append(y if x == 1 else x)
    return tuple(reversed(out))
```

The rule against `np.broadcast_shapes`

```
$ uv run -m u1.broadcast
        (3, 4) with (4,)       → (3, 4)
        (3, 1) with (1, 4)     → (3, 4)
     (2, 3, 4) with (3, 1)     → (2, 3, 4)
  (8, 1, 6, 1) with (7, 1, 5)  → (8, 7, 6, 5)
        (3, 4) with (3,)       → error
        (2, 1) with (8, 4, 3)  → error
```

Broadcasting replaces loops with whole-array expressions, which run in compiled code. It can also hide a large cost. Consider the table of squared distances between 1,000 points and 1,000 others, in 64 dimensions, which nearest-neighbour search and several later chapters need. The program computes it three ways:

Squared distances by loops, by broadcasting, and by expanding the square — code/u1/broadcast.py

```python
def distances_loops(x: np.ndarray, y: np.ndarray) -> np.ndarray:
    out = np.empty((len(x), len(y)), dtype=x.dtype)
    for i in range(len(x)):
        for j in range(len(y)):
            out[i, j] = np.sum((x[i] - y[j]) ** 2)
    return out


def distances_broadcast(x: np.ndarray, y: np.ndarray) -> np.ndarray:
    return ((x[:, None, :] - y[None, :, :]) ** 2).sum(axis=-1)


def distances_matmul(x: np.ndarray, y: np.ndarray) -> np.ndarray:
    return (x * x).sum(1)[:, None] + (y * y).sum(1)[None, :] - 2 * x @ y.T
```

The broadcast version builds `x[:, None, :] - y[None, :, :]`, a tensor of shape $(1000, 1000, 64)$, before summing it away. The third uses the identity

$$\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2x \cdot y,$$

which turns the whole table into one matrix multiplication and two small sums.

The three versions on the M2 Pro

```
1000 x 1000 squared distances, 64 dimensions:
  Python loops (estimated from 100 x 100)      1595 ms
  broadcasting                                 35.0 ms, 244 MiB temporary
  |x|^2 + |y|^2 - 2 x.y                        0.5 ms
```

The loops spend their time in the Python interpreter, and the broadcast version spends it creating and reading a 244 MiB temporary. The matrix multiplication does the same arithmetic as the broadcast version with almost no memory traffic. Expanding the square has one drawback. When x and y are close and large, the subtraction cancels and the result can come out slightly negative, so code that takes its square root clamps it at zero first.

PITFALL

Broadcasting raises an error when shapes are incompatible, but it can also succeed with a shape you did not intend. Subtracting a vector of shape `(n,)` from a column of shape `(n, 1)` gives an $n \times n$ matrix instead of n differences. When a loss or a metric comes out with an unexpected value, print the shapes of every intermediate first.

Counting the cost

Two numbers describe the cost of an operation: the floating-point operations it performs, its *FLOPs*, counting a multiplication and an addition as two; and the bytes it must move between memory and the processor. Their ratio is the operation's *arithmetic intensity*, in FLOPs per byte.

Multiplying an $n \times k$ matrix by a $k \times m$ matrix computes nm dot products of length k: $2nkm$ FLOPs. It must at least read both inputs and write the output, $4(nk + km + nm)$ bytes in `float32`. For square matrices that is $2n^3$ FLOPs over $12n^2$ bytes, an intensity of $\frac{n}{6}$, which grows with the matrix. Adding two vectors of length n performs n FLOPs and moves $12n$ bytes, an intensity of $\frac{1}{12}$, whatever n.

Every processor has a peak rate of arithmetic and a peak rate of memory traffic. An operation whose intensity is below the ratio of the two is limited by memory, and an operation above it by arithmetic. Large matrix multiplications are among the few operations in machine learning that can keep a processor's arithmetic busy, which is why models are built from them, and why so much engineering goes into making everything else look like one. Unit 4 makes this picture, the *roofline*, precise.

From loops to the machine's peak

The program multiplies square matrices four ways and checks every result against a float64 reference:

The definition, in Python — code/u1/matmul.py

```python
def matmul_loops(a: list[list[float]], b: list[list[float]]) -> list[list[float]]:
    n, k, m = len(a), len(b), len(b[0])
    c = [[0.0] * m for _ in range(n)]
    for i in range(n):
        for j in range(m):
            total = 0.0
            for p in range(k):
                total += a[i][p] * b[p][j]
            c[i][j] = total
    return c
```

The same product as a sum of k outer products, each a whole-array NumPy operation — code/u1/matmul.py

```python
def matmul_outer(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    c = np.zeros((a.shape[0], b.shape[1]), dtype=a.dtype)
    for p in range(a.shape[1]):
        c += np.outer(a[:, p], b[p])
    return c
```

The outer-product version runs its arithmetic in compiled code, but each of its k steps reads and writes the whole $n \times n$ output, so its intensity is about $\frac{1}{4}$, whatever n. The third is NumPy's `a @ b`, which calls a

BLAS library: on macOS, Apple's Accelerate, which on Apple Silicon uses the CPU's matrix coprocessor. The fourth runs the product on the GPU through PyTorch.

Four matrix multiplications on the M2 Pro

```
$ uv run -m u1.matmul
Python loops              n=64          6.795 ms       0.1 GFLOP/s
NumPy outer products      n=64          0.175 ms       3.0 GFLOP/s
NumPy a @ b (BLAS)        n=64          0.002 ms     306.8 GFLOP/s
PyTorch on mps            n=64          0.480 ms       1.1 GFLOP/s
Python loops              n=128        55.486 ms       0.1 GFLOP/s
NumPy outer products      n=128         1.043 ms       4.0 GFLOP/s
NumPy a @ b (BLAS)        n=128         0.005 ms     783.3 GFLOP/s
PyTorch on mps            n=128         0.504 ms       8.3 GFLOP/s
NumPy outer products      n=256         5.560 ms       6.0 GFLOP/s
NumPy a @ b (BLAS)        n=256         0.029 ms    1146.3 GFLOP/s
PyTorch on mps            n=256         0.504 ms      66.6 GFLOP/s
NumPy outer products      n=512        38.247 ms       7.0 GFLOP/s
NumPy a @ b (BLAS)        n=512         0.205 ms    1309.0 GFLOP/s
PyTorch on mps            n=512         0.686 ms     391.3 GFLOP/s
NumPy outer products      n=1024      317.339 ms       6.8 GFLOP/s
NumPy a @ b (BLAS)        n=1024        0.794 ms    2705.0 GFLOP/s
PyTorch on mps            n=1024        1.079 ms    1990.5 GFLOP/s
NumPy outer products      n=2048     2538.482 ms       6.8 GFLOP/s
NumPy a @ b (BLAS)        n=2048        8.768 ms    1959.4 GFLOP/s
PyTorch on mps            n=2048        3.307 ms    5195.1 GFLOP/s
NumPy a @ b (BLAS)        n=4096       63.149 ms    2176.4 GFLOP/s
PyTorch on mps            n=4096       24.753 ms    5552.4 GFLOP/s
```

The speeds span more than four orders of magnitude (Figure 1.1.1). Python loops manage about a tenth of a GFLOP/s, because each multiply-add costs dozens of interpreted instructions. The outer products reach a few GFLOP/s, held down by memory traffic as their intensity predicts. BLAS passes a trillion FLOPs per second once n is a few hundred. It uses the CPU's matrix units and *blocking*, splitting the matrices into tiles small enough to stay in cache so that each number fetched from memory is used many times. The GPU is slower than the CPU for small matrices, because sending it work and waiting for the result costs about half a millisecond here, and fastest for large ones, at over 5 TFLOP/s. Neural networks follow the same pattern: overheads dominate small models, arithmetic dominates large ones, and memory traffic dominates those in between.

KEY IDEA

Before asking how to make an operation faster, count its FLOPs and its bytes. An operation that moves many bytes per FLOP can only be made faster by moving fewer bytes; one that performs

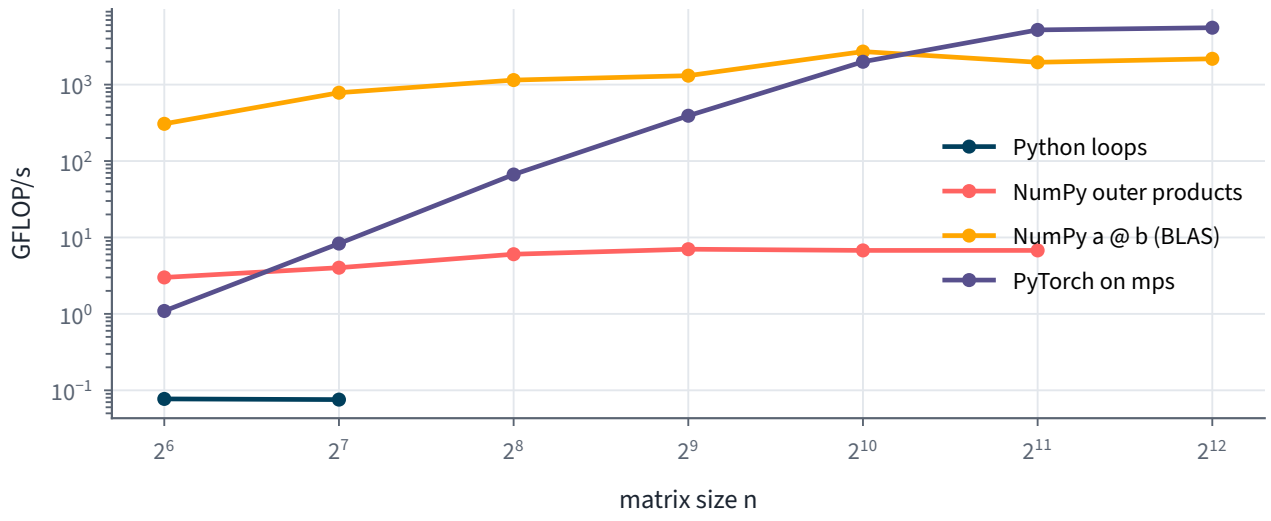


Figure 1.1.1. Speed of the four methods against matrix size, on a logarithmic scale.

many FLOPs per byte is limited by arithmetic, and the best available library is usually already close to the limit.

IN PYTORCH

A `torch.Tensor` is the same recipe over a buffer, with one addition, the device the buffer lives on. `x.stride()` reports strides in elements rather than bytes, `x.is_contiguous()` and `x.contiguous()` behave as in NumPy, and `x.view(shape)` reshapes only when no copy is needed, raising an error otherwise. `torch.from_numpy(a)` shares `a`'s memory; `x.to("mps")` copies to the GPU. Broadcasting follows the same rule.

Reading

- The NumPy documentation, "The N-dimensional array (ndarray)" and "Broadcasting".
- Ulrich Drepper, "What Every Programmer Should Know About Memory", 2007, sections 3 and 6: caches, and how to write code that uses them.
- Kazushige Goto and Robert van de Geijn, "Anatomy of High-Performance Matrix Multiplication", *ACM Transactions on Mathematical Software* 34(3), 2008.

Labs

LAB 1.1.1 Strides by hand — 1 hour

Goal. Predict views before NumPy shows them, and see a view's writes reach the original.

Code. `code/u1/strides.py`

1. For a `float32` array of shape `(2, 3, 4)`, write down the strides of `a`, `a.transpose(2, 0, 1)`, `a[:, ::2, 1:]`, `a[..., None]` and `a.reshape(6, 4)`, then check them in Python.
2. Write `offset(index, strides)` and use it, with `np.frombuffer`, to read three elements of a transposed view straight from the original buffer.
3. Set every element of `a[:, ::2]` to zero and print `a`. Then do the same through `a.T.reshape(-1)` and explain the difference.

DONE WHEN

- [ ] All five predictions match NumPy, or you can explain each one that did not.
- [ ] Your `offset` reads the right elements, and you can say which of the two writes changed `a` and why.

EVIDENCE FOR THE NOTEBOOK

- The predictions, the code and the printed arrays.

LAB 1.1.2 Broadcasting drills — 1 hour

Goal. Write common operations as single broadcast expressions.

Code. `code/u1/broadcast.py`

1. Without loops, compute: the outer product of two vectors; every row of a matrix minus its row mean; a batch of 32 vectors each multiplied by its own 8×8 matrix; and a mask of shape (n, n) that is true above the diagonal.
2. Check each against a loop version on random inputs.
3. Find one case in which broadcasting produces a result of the wrong shape without an error, and write the assertion that would have caught it.

DONE WHEN

- [ ] Each expression matches its loop version.
- [ ] Your silent-shape example and its assertion are in the notebook.

EVIDENCE FOR THE NOTEBOOK

- The four expressions with their checks, and the example.

LAB 1.1.3 What a temporary costs — 1 hour

Goal. Measure the memory and time of the three distance computations as the problem grows.

Code. `code/u1/broadcast.py`

1. Time the broadcast and matrix-multiplication versions for 1,000, 2,000 and 4,000 points in 64 dimensions, watching memory use in Activity Monitor or `top`.
2. Find the size at which the broadcast version no longer fits comfortably in your machine's memory.
3. Construct two nearby points with large coordinates for which the expanded square comes out negative, and fix the function.

DONE WHEN

- [ ] You have both timings at three sizes and the size at which broadcasting runs out of memory.
- [ ] Your fixed function never returns a negative distance.

EVIDENCE FOR THE NOTEBOOK

- A table of times and peak memory, and the fix.

LAB 1.1.4 Matrix multiplication on your machine — 1.5 hours

Goal. Measure the four multiplications on your own machine and explain each with FLOPs and bytes.

Code. `code/u1/matmul.py`

1. Run `uv run -m u1.matmul` with the default sizes, and once more with `--sizes 32 64 96`.
2. For each method at $n = 512$, compute its arithmetic intensity and say whether memory or arithmetic limits it.
3. Find your CPU's and GPU's published peaks and express the best measured speed of each as a fraction of them.

DONE WHEN

- [ ] Your table has every method at every size it runs at.
- [ ] For each method you can say what limits it, from its intensity and your measurements.

EVIDENCE FOR THE NOTEBOOK

- The table, a plot of it, and a paragraph per method.

Problem set

1. A `float32` tensor of shape $(32, 3, 224, 224)$ is stored contiguously. What are its strides in bytes, and at what offset is element $(5, 2, 10, 20)$?
2. Which of `a[1:, :]`, `a[:, 1:]` and `a[::-1]` of a contiguous 4×4 array is contiguous? Can a stride be negative?
3. Broadcasting `x` of shape $(n, 1)$ against `y` of shape $(m,)$ gives an $n \times m$ result. Write the stride NumPy uses internally to read `y` along the first dimension.

4. Count the FLOPs and minimum bytes for the product of a 4096×4096 matrix with a vector. Is it limited by memory or by arithmetic on a processor with 200 GB/s and 6 TFLOP/s?
5. The outer-product matmul reaches about 7 GFLOP/s. Using its intensity of $\frac{1}{4}$ FLOP per byte, what memory bandwidth does that imply, and how does it compare with the 88 GB/s copy measured in this chapter?

COMPLETION CHECKLIST

- ☐ I can explain shape, strides and contiguity, and say whether an operation returns a view or a copy.
- ☐ I can apply the broadcasting rule by hand and spot a broadcast that gives the wrong shape.
- ☐ I can count the FLOPs and bytes of an operation and say what limits it.
- ☐ I can explain why BLAS is fast and when a GPU is slower than a CPU.
- ☐ Lab 1.1.1–1.1.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.2

Gradients by machine

TIME	7 hours
YOU BUILD	A reverse-mode automatic differentiation engine for single numbers; then `tiny`, a tensor library of about 650 lines whose gradients respect broadcasting, checked operation by operation against central differences and, for a whole network, against PyTorch to within float64 rounding.
NEEDS	Chapter 1.1. Derivatives and the chain rule as in the Handbook's maths part.

WHY IT MATTERS

Training a model means nudging its parameters in the direction that lowers a loss, and that direction comes from the gradient, the derivative of the loss with respect to every parameter. A language model has hundreds of millions of parameters, and its gradient must be computed millions of times. *Automatic differentiation* computes all of it exactly, at the cost of a small multiple of computing the loss itself. Every framework is built on it. This chapter builds one, first for single numbers and then for tensors, so that you know how every gradient in later chapters is computed, and where to look when one is wrong.

SKIP TEST

1. For $d = \tanh(ab + e^a) \cdot b$, draw the computation graph and compute $\partial d/\partial a$ with the chain rule.
2. A loss depends on a million parameters. Why is computing its gradient backwards from the loss cheaper than forwards from the parameters?
3. `y = x + b` with `x` of shape $(32, 10)$ and `b` of shape $(10,)$. What is the shape of $\partial L/\partial b$, and how is it computed from $\partial L/\partial y$?
4. Why does a backward pass add into each gradient instead of assigning it?

If all four are easy, read the sections on broadcasting and checking, and do Labs 1.2.2 and 1.2.3.

Core ideas

Derivatives on a graph

Any computation can be drawn as a graph: inputs at the leaves, each operation a node whose value depends on its inputs, and the result at the root. Take

$$c = ab + e^a, \quad d = \tanh(c) \cdot b.$$

The *chain rule* says how a change in a reaches d: through every path from a to d, multiplying the local derivative of each edge along the path and adding the paths. Here a reaches d through ab and through e^a, both feeding c, so

$$\frac{\partial d}{\partial a} = \frac{\partial d}{\partial c} \cdot (b + e^a), \quad \frac{\partial d}{\partial c} = b\big(1 - \tanh^2 c\big).$$

b reaches d twice as well, through c and directly. Write $\bar{v}$ for $\partial d/\partial v$, the *adjoint* of v: how much the result changes per unit change in v. The adjoint of the root is 1. For any other node, its adjoint is the sum, over the nodes that use it, of their adjoint times the local derivative of the use. Computing adjoints from the root towards the leaves, each node visited after every node that uses it, gives the derivative of the result with respect to every node in one sweep. This is *reverse-mode automatic differentiation*, and in machine learning, where the result is a single loss, it is called *backpropagation*.

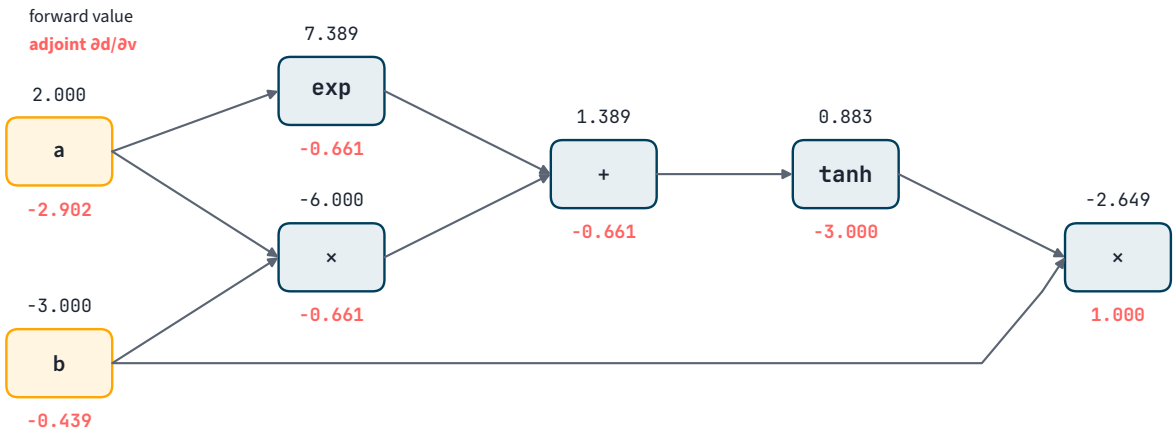


Figure 1.2.1. The computation of d as a graph. Each node shows its value from the forward pass, above, and its adjoint from the backward pass, below.

A scalar engine

A small class carries out this procedure for single numbers. Each `Value` holds its number, its adjoint, the values it was computed from, and a function that pushes its adjoint to them:

Addition and multiplication, each recording how to send gradients back to its inputs — code/aif/tiny/scalar.py

```
class Value:
    def __init__(self, data: float, parents: tuple = (), op: str = ""):
        self.data = float(data)
        self.grad = 0.0
        self.parents = parents
        self.op = op
        self.backward_step = lambda: None

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")

        def step():
            self.grad += out.grad
            other.grad += out.grad

        out.backward_step = step
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")

        def step():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad

        out.backward_step = step
        return out
```

The function attached by `__mul__` encodes the local derivatives of $z = xy$: $\partial z/\partial x = y$ and $\partial z/\partial y = x$, each multiplied by the adjoint arriving from above. Unary operations follow the same pattern:

Exponential, logarithm and tanh, each with its derivative — code/aif/tiny/scalar.py

```
def exp(self):
    out = Value(math.exp(self.data), (self,), "exp")

    def step():
        self.grad += out.data * out.grad

    out.backward_step = step
```

```
    return out

def log(self):
    out = Value(math.log(self.data), (self,), "log")

    def step():
        self.grad += out.grad / self.data

    out.backward_step = step
    return out

def tanh(self):
    out = Value(math.tanh(self.data), (self,), "tanh")

    def step():
        self.grad += (1 - out.data**2) * out.grad

    out.backward_step = step
    return out
```

`backward` orders the graph so that every node comes after the nodes it was computed from, then walks that order in reverse:

Topological order by depth-first search, then the adjoints in reverse — code/aif/tiny/scalar.py

```
def backward(self) -> None:
    order, seen, stack = [], set(), [(self, False)]
    while stack:
        node, expanded = stack.pop()
        if expanded:
            order.append(node)
        elif id(node) not in seen:
            seen.add(id(node))
            stack.append((node, True))
            stack.extend((p, False) for p in node.parents)
    self.grad = 1.0
    for node in reversed(order):
        node.backward_step()
```

The search uses an explicit stack, since a graph with tens of thousands of nodes would exceed Python's recursion limit. The gradients are *accumulated* with `+=`: a value used twice, as b is, receives one contribution from each use, which is the sum over paths that the chain rule requires.

The example, built from `Value`s — code/u1/gradcheck.py

```
a, b = Value(2.0), Value(-3.0)
c = a * b + a.exp()
d = c.tanh() * b
d.backward()
```

The engine's derivatives agree with the chain rule worked by hand

```
$ uv run -m u1.gradcheck
scalar engine: d = -2.648889, dd/da = -2.901731, dd/db = -0.439294 (match the chain rule)
```

KEY IDEA

Reverse mode costs one backward sweep per *output*, whatever the number of inputs. Each node's backward step costs about as much as its forward step, so the whole gradient of a scalar loss costs a small multiple, typically two to three times, of computing the loss. Forward mode, which carries derivatives from the inputs towards the output, would cost one sweep per *input*: a million sweeps for a million parameters.

From numbers to tensors

A graph of single numbers would hold billions of nodes for a modest network. Real engines make each node a whole tensor operation: a matrix multiplication, an element-wise function, a sum. As before, each operation has a backward rule that turns the adjoint of its output, a tensor of the output's shape, into adjoints of its inputs, tensors of the inputs' shapes.

For element-wise operations the rule is the scalar rule applied at every position: for $y = \exp(x)$, $\overline{x} = \overline{y} \odot \exp(x)$, with $\odot$ the element-wise product. Matrix multiplication needs a short derivation.

DERIVATION

Let $C = AB$ with A of shape $n \times k$ and B of shape $k \times m$, so $C_{ij} = \sum_p A_{ip} B_{pj}$. The loss depends on A_{ip} through every C_{ij} in row i, with $\partial C_{ij} / \partial A_{ip} = B_{pj}$. Summing over those uses,

$$\overline{A}_{ip} = \sum_j \overline{C}_{ij} B_{pj} = \left(\overline{C} B^T\right)_{ip},$$

and by the same argument $\overline{B} = A^T \overline{C}$. Both are matrix multiplications, so the backward pass of a matrix multiplication costs twice its forward pass, and a network's training step about three times its forward pass.

Reductions run the other way: if $s = \sum_i x_i$, each x_i affects s with derivative 1, so $\overline{x}$ is $\overline{s}$ copied to every position. Indexing gathers some elements of a tensor; its adjoint scatters the incoming gradient back to the positions it came from, adding where an index repeats. Broadcasting needs more care. When `b` of shape $(10,)$ is added to every row of `x` of shape $(32, 10)$, each element of `b` is used 32 times, so its adjoint is the sum of the 32 adjoints of its uses. In general, the adjoint of a broadcast is a sum over the dimensions it stretched:

Undoing a broadcast: sum away the dimensions that were added or stretched — code/aif/tiny/tensor.py

```
def unbroadcast(grad: np.ndarray, shape: tuple) → np.ndarray:
    while grad.ndim > len(shape):
        grad = grad.sum(axis=0)
    for axis, size in enumerate(shape):
        if size == 1 and grad.shape[axis] ≠ 1:
            grad = grad.sum(axis=axis, keepdims=True)
    return grad
```

tiny

`tiny` is a tensor library with exactly these rules. A `Tensor` holds a NumPy array, a gradient, its parents and a backward step. Operations create their result with `child`, which links the graph only if some input needs a gradient:

The tensor: data, gradient, parents and a backward step — code/aif/tiny/tensor.py

```
class Tensor:
    def __init__(self, data, requires_grad: bool = False):
        array = np.asarray(data)
        self.data = array if array.dtype.kind == "f" else array.astype(np.float64)
        self.requires_grad = requires_grad
        self.grad: np.ndarray | None = None
        self.parents: tuple = ()
        self.backward_step = None

    def child(self, data: np.ndarray, parents: tuple) → "Tensor":
        out = Tensor(data)
        if _grad_enabled and any(p.requires_grad for p in parents):
            out.requires_grad = True
            out.parents = parents
        return out

    def accumulate(self, grad: np.ndarray) → None:
        if self.requires_grad:
            self.grad = grad if self.grad is None else self.grad + grad

    def wrap(self, other) → "Tensor":
```

```
        return (
            other
            if isinstance(other, Tensor)
            else Tensor(np.asarray(other, dtype=self.data.dtype))
        )
```

`requires_grad` marks the tensors that need gradients, the parameters. Inputs and labels do not need gradients, and a node computed only from them does not need one either; skipping them saves both time and memory. `no_grad` switches graph building off entirely, for evaluation.

Addition, multiplication and matrix multiplication, each undoing broadcasting on the way back — code/aif/tiny/tensor.py

```
def __add__(self, other) -> "Tensor":
    other = self.wrap(other)
    out = self.child(self.data + other.data, (self, other))

    def step():
        self.accumulate(unbroadcast(out.grad, self.shape))
        other.accumulate(unbroadcast(out.grad, other.shape))

    out.backward_step = step
    return out

def __mul__(self, other) -> "Tensor":
    other = self.wrap(other)
    out = self.child(self.data * other.data, (self, other))

    def step():
        self.accumulate(unbroadcast(out.grad * other.data, self.shape))
        other.accumulate(unbroadcast(out.grad * self.data, other.shape))

    out.backward_step = step
    return out

def __matmul__(self, other) -> "Tensor":
    out = self.child(self.data @ other.data, (self, other))

    def step():
        self.accumulate(unbroadcast(out.grad @ other.data.swapaxes(-1, -2), self.shape))
        other.accumulate(unbroadcast(self.data.swapaxes(-1, -2) @ out.grad, other.shape))

    out.backward_step = step
    return out
```

Sum, mean and max. The gradient of a max goes to the winning elements, shared equally among ties — code/aif/tiny/tensor.py

```
def sum(self, axis=None, keepdims: bool = False) -> "Tensor":
    out = self.child(self.data.sum(axis=axis, keepdims=keepdims), (self,))

    def step():
        grad = out.grad
        if axis is not None and not keepdims:
            grad = np.expand_dims(grad, axis)
        self.accumulate(np.broadcast_to(grad, self.shape).copy())

    out.backward_step = step
    return out

def mean(self, axis=None, keepdims: bool = False) -> "Tensor":
    count = self.data.size // np.asarray(self.data.sum(axis=axis, keepdims=keepdims)).size
    return self.sum(axis, keepdims) / count

def max(self, axis=None, keepdims: bool = False) -> "Tensor":
    out = self.child(self.data.max(axis=axis, keepdims=keepdims), (self,))

    def step():
        peak = self.data.max(axis=axis, keepdims=True)
        winners = (self.data == peak).astype(self.data.dtype)
```

```
        winners /= winners.sum(axis=axis, keepdims=True)
        grad = out.grad if keepdims or axis is None else np.expand_dims(out.grad, axis)
        self.accumulate(winners * grad)

    out.backward_step = step
    return out
```

Reshape and transpose move gradients back into the original layout; indexing scatters them — code/aif/tiny/tensor.py

```
def reshape(self, *shape) -> "Tensor":
    out = self.child(self.data.reshape(*shape), (self,))

    def step():
        self.accumulate(out.grad.reshape(self.shape))

    out.backward_step = step
    return out

def transpose(self, *axes) -> "Tensor":
    axes = axes or tuple(reversed(range(self.ndim)))
    out = self.child(self.data.transpose(axes), (self,))

    def step():
        self.accumulate(out.grad.transpose(np.argsort(axes)))

    out.backward_step = step
    return out

def __getitem__(self, index) -> "Tensor":
    out = self.child(self.data[index], (self,))

    def step():
        grad = np.zeros_like(self.data)
        np.add.at(grad, index, out.grad)
        self.accumulate(grad)

    out.backward_step = step
    return out
```

`backward` is the scalar engine's, visiting only nodes that need gradients:

The tensor engine's backward pass — code/aif/tiny/tensor.py

```
def backward(self, grad: np.ndarray | None = None) -> None:
    order, seen, stack = [], set(), [(self, False)]
    while stack:
        node, expanded = stack.pop()
        if expanded:
            order.append(node)
        elif id(node) not in seen:
            seen.add(id(node))
            stack.append((node, True))
            stack.extend((p, False) for p in node.parents if p.requires_grad)
    self.grad = np.ones_like(self.data) if grad is None else grad
    for node in reversed(order):
        if node.backward_step is not None:
            node.backward_step()
```

One operation is written as a single node for numerical reasons, the logarithm of the softmax, which Chapter 1.3 derives. Its backward step uses the result of the forward step instead of recomputing anything:

Log-softmax as one node, and the cross-entropy loss built from it — code/aif/tiny/tensor.py

```
def log_softmax(x: Tensor, axis: int = -1) → Tensor:
    shifted = x.data - x.data.max(axis=axis, keepdims=True)
    out = x.child(shifted - np.log(np.exp(shifted).sum(axis=axis, keepdims=True)), (x,))

    def step():
        softmax = np.exp(out.data)
        x.accumulate(out.grad - softmax * out.grad.sum(axis=axis, keepdims=True))

    out.backward_step = step
    return out


def cross_entropy(logits: Tensor, targets: np.ndarray) → Tensor:
    picked = log_softmax(logits)[np.arange(len(targets)), targets]
    return -picked.mean()
```

PITFALL

A backward step reads tensors saved by the forward pass: `__mul__` reads `self.data` and `other.data` when the gradient arrives. If anything modifies those arrays in place between the forward and backward passes, the gradient is silently wrong. In `tiny`, as in PyTorch, an optimiser may update parameters in place only after `backward` has finished.

Checking gradients

A wrong gradient rarely causes an error, and the model merely trains badly, so every gradient should be checked once against an independent calculation. The *central difference*

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h}$$

has an error proportional to h^2, so in float64 with $h = 10^{-6}$ it agrees with the true derivative to about eight digits. It needs two evaluations of f per input, which is why it is used to check gradients and never to compute them for training.

Central differences, one input element at a time — code/u1/gradcheck.py

```
def numeric_grad(f, x: np.ndarray, h: float = 1e-6) → np.ndarray:
    grad = np.zeros_like(x)
    for i in np.ndindex(x.shape):
        saved = x[i]
        x[i] = saved + h
        up = f()
        x[i] = saved - h
        down = f()
        x[i] = saved
        grad[i] = (up - down) / (2 * h)
    return grad
```

Comparing analytic and numeric gradients by relative error — code/u1/gradcheck.py

```
def gradcheck(
    name: str, fn, *shapes, rng: np.random.Generator, positive: bool = False
) → float:
    arrays = [rng.uniform(0.5, 2.0, s) if positive else rng.normal(size=s) for s in shapes]
    inputs = [Tensor(a, requires_grad=True) for a in arrays]
    fn(*inputs).sum().backward()
    worst = 0.0
    for t, a in zip(inputs, arrays):
        numeric = numeric_grad(lambda: fn(*[Tensor(b) for b in arrays]).sum().item(), a)
        scale = np.maximum(np.abs(numeric) + np.abs(t.grad), 1e-8)
        worst = max(worst, float(np.max(np.abs(t.grad - numeric) / scale)))
    check.expect(name, worst < 1e-6, f"relative error {worst:.2e}")
    print(f"  {name:<34} {worst:9.1e}")
    return worst
```

The comparison uses the relative error, the difference divided by the sizes of the two values, since a difference of 10^{-6} is negligible for a gradient of 1,000 and alarming for one of 10^{-6}. Inputs to `log`, `**` and division are kept positive, and the test cases include broadcasting in every operation that allows it:

The fifteen cases — code/u1/gradcheck.py

```
gradcheck("add, broadcast (3,4)+(4,)", lambda x, y: x + y, (3, 4), (4,), rng=rng)
gradcheck("mul, broadcast (3,1)*(1,4)", lambda x, y: x * y, (3, 1), (1, 4), rng=rng)
gradcheck("matmul (3,5)@(5,2)", lambda x, y: x @ y, (3, 5), (5, 2), rng=rng)
gradcheck("batched matmul (2,3,4)@(4,5)", lambda x, y: x @ y, (2, 3, 4), (4, 5), rng=rng)
gradcheck(
    "divide, broadcast (3,4)/(3,1)",
    lambda x, y: x / y,
    (3, 4),
    (3, 1),
    rng=rng,
    positive=True,
)
gradcheck("power 1.5", lambda x: x**1.5, (3, 4), rng=rng, positive=True)
gradcheck("exp, log", lambda x: x.exp() + x.log(), (3, 4), rng=rng, positive=True)
gradcheck(
    "relu, tanh, sigmoid", lambda x: x.relu() + x.tanh() + x.sigmoid(), (3, 4), rng=rng
)
gradcheck("sum over axis 0", lambda x: x.sum(axis=0) * 2.0, (3, 4), rng=rng)
gradcheck("mean keepdims", lambda x: x * x.mean(axis=1, keepdims=True), (3, 4), rng=rng)
gradcheck("max over axis 1", lambda x: x.max(axis=1), (3, 4), rng=rng)
gradcheck("reshape, transpose", lambda x: x.reshape(4, 3) @ x + x.T @ x, (3, 4), rng=rng)
gradcheck("index rows [0, 2, 2]", lambda x: x[np.array([0, 2, 2])], (3, 4), rng=rng)
gradcheck(
    "log_softmax", lambda x: log_softmax(x) * Tensor(np.arange(4.0)), (3, 4), rng=rng
)
gradcheck(
    "cross_entropy", lambda x: cross_entropy(x, np.array([1, 0, 3])), (3, 4), rng=rng
)
```

Every operation within 10^{-6} of central differences

```
$ uv run -m u1.gradcheck
tensor operations against central differences (float64):
  operation                          rel. error
  add, broadcast (3,4)+(4,)            3.7e-10
  mul, broadcast (3,1)*(1,4)           1.3e-10
  matmul (3,5)@(5,2)                   5.3e-09
  batched matmul (2,3,4)@(4,5)         1.4e-08
  divide, broadcast (3,4)/(3,1)        7.7e-10
  power 1.5                            5.4e-10
  exp, log                             6.6e-10
  relu, tanh, sigmoid                  4.8e-09
  sum over axis 0                      7.0e-11
  mean keepdims                        1.7e-10
  max over axis 1                      1.5e-10
  reshape, transpose                   6.9e-10
  index rows [0, 2, 2]                 1.5e-10
  log_softmax                          3.4e-09
  cross_entropy                        1.1e-08
```

A final check computes the gradient of a small two-layer network with `tiny` and with PyTorch, from the same weights and data:

The same network and loss in both libraries — code/u1/gradcheck.py

```
x = rng.normal(size=(16, 8))
targets = rng.integers(0, 5, 16)
w1, b1 = rng.normal(size=(8, 32)) * 0.3, np.zeros(32)
w2, b2 = rng.normal(size=(32, 5)) * 0.3, np.zeros(5)

params = [Tensor(a, requires_grad=True) for a in (w1, b1, w2, b2)]
loss = cross_entropy(
    (Tensor(x) @ params[0] + params[1]).tanh() @ params[2] + params[3], targets
)
loss.backward()

ref = [torch.tensor(a, requires_grad=True) for a in (w1, b1, w2, b2)]
logits = torch.tanh(torch.tensor(x) @ ref[0] + ref[1]) @ ref[2] + ref[3]
ref_loss = torch.nn.functional.cross_entropy(logits, torch.tensor(targets))
ref_loss.backward()
```

`tiny` and PyTorch agree to within float64 rounding

```
two-layer net vs PyTorch: loss 1.734024382277 vs 1.734024382277, gradients within 4.2e-17
```

The gradients differ by less than 10^{-16}, which is float64 rounding, because the two libraries perform the same arithmetic in nearly the same order.

IN PYTORCH

PyTorch's autograd follows the same design. A tensor with `requires_grad=True` records each operation in a `grad_fn` node; `loss.backward()` walks the graph in reverse, accumulating into each leaf's `.grad`; `torch.no_grad()` disables recording. Gradients accumulate across calls, which is why training loops call `optimizer.zero_grad()` before each `backward`. `torch.autograd.gradcheck` performs this chapter's check on any function, and custom operations subclass `torch.autograd.Function`, supplying `forward` and `backward` as `tiny`'s operations do.

Reading

- Atılım Güneş Baydin, Barak Pearlmutter, Alexey Radul and Jeffrey Siskind, "Automatic Differentiation in Machine Learning: a Survey", *Journal of Machine Learning Research* 18, 2018.
- The PyTorch documentation, "Autograd mechanics".
- Andreas Griewank and Andrea Walther, *Evaluating Derivatives*, second edition, SIAM, 2008, chapters 3 and 4, for the theory of forward and reverse modes.

Labs

LAB 1.2.1 XOR with a scalar engine — 1.5 hours

Goal. Train a network built from `Value`s, and see why one layer is not enough.

Code. `code/aif/tiny/scalar.py`

1. Add `sigmoid` to `Value`, with its derivative, and check it against central differences.
2. Build a network with two inputs, two hidden units using `tanh`, and one sigmoid output, from `Value`s. Train it on the four XOR examples with gradient descent on the squared error.
3. Try again with no hidden layer and explain the result.

DONE WHEN

- [ ] The two-layer network classifies all four XOR examples correctly from at least four of five random initialisations.

- [ ] You can explain, with a drawing, why no single-layer network can.

EVIDENCE FOR THE NOTEBOOK
- The training code, the loss curve, and the drawing.

LAB 1.2.2 A new operation — 1.5 hours

Goal. Extend `tiny` with operations of your own and check their gradients.

Code. `code/aif/tiny/tensor.py`

1. Add `concat(tensors, axis)` and `where(condition, a, b)`, each with a backward step.
2. Add both to the cases in `u1/gradcheck.py`, including inputs that broadcast in `where`.
3. Add `softplus`, $\log(1 + e^x)$, written so that it neither overflows for large x nor loses precision for very negative x, and check it at $x = \pm 50$.

DONE WHEN
- [ ] All three operations pass the gradient check.
- [ ] `softplus` returns finite, accurate values and gradients at ± 50.

EVIDENCE FOR THE NOTEBOOK
- The code and the extended checker's output.

LAB 1.2.3 Break it on purpose — 1 hour

Goal. Learn what wrong gradients look like before you meet one by accident.

Code. `code/u1/gradcheck.py`

1. Make `accumulate` assign instead of add. Run the checker and find which cases fail, and why exactly those.
2. Restore it, then remove the call to `unbroadcast` in `__add__`. Note the error that results.
3. Finally, make an optimiser step run before `backward` in a two-step training loop, and describe the symptom.

DONE WHEN
- [ ] For each break, you can name the failing cases and explain the mechanism.

EVIDENCE FOR THE NOTEBOOK
- The checker's output for each break, and the explanations.

LAB 1.2.4 What a gradient costs — 1.5 hours

Goal. Measure the time and memory of backward passes.

Code. `code/aif/tiny/nn.py`

1. Time the forward pass alone and the forward and backward passes together for an MLP of widths 784, 1024, 1024, 10 in `tiny`, at batch sizes 32, 128 and 512.
2. Count the tensors that the graph holds at the moment `backward` begins, and their total size in bytes.
3. Explain how the memory held would grow with depth and with batch size.

DONE WHEN
- [ ] You have the ratio of backward to forward time at each batch size, and can relate it to the matrix-multiplication rule.

- [ ] Your byte count for the graph matches a prediction made from the layer sizes.

EVIDENCE FOR THE NOTEBOOK

- The table of times, the count, and the prediction.

Problem set

1. Derive the backward step for $y = x/\|x\|$, where x is a vector, and simplify it.
2. A node's output is used by three later nodes. Show that adding the three contributions to its adjoint is exactly the multivariable chain rule.
3. Forward-mode differentiation propagates a derivative alongside each value. For a function from $\mathbb{R}^n$ to $\mathbb{R}^m$, how many passes do forward and reverse mode need for the full Jacobian? When is forward mode the better choice?
4. The central difference with $h = 10^{-6}$ is accurate to about 10^{-10} in float64. Why would $h = 10^{-12}$ be worse, and what would happen in float32?
5. `max` shares its gradient equally among tied elements. PyTorch's `max` along a dimension sends it to one of them. Which choice is a valid subgradient, and does it matter for training?

COMPLETION CHECKLIST

- [ ] I can draw a computation graph and compute every adjoint by hand.
- [ ] I can explain why reverse mode suits a scalar loss, and what the backward pass costs.
- [ ] I can derive the backward steps of matrix multiplication, reductions, broadcasting and indexing.
- [ ] I can check any gradient against central differences and judge the result.
- [ ] Lab 1.2.1–1.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.3

Models as probability

TIME	5 hours
YOU BUILD	Linear regression solved in closed form, by gradient descent and by maximum likelihood with a learned noise level, all three agreeing to four places; softmax regression on MNIST at 92%, with its weights drawn as images; and a softmax and cross-entropy that survive any logits.
NEEDS	Chapter 1.2. Probability as in the Handbook's maths part: distributions, expectations, independence.

WHY IT MATTERS

A loss function can look like an arbitrary choice, squared error for one problem and cross-entropy for another. Each standard loss follows from one assumption about how the data were generated and from one principle, that a model should make the observed data as probable as possible. Seeing losses this way explains when each is appropriate, what its value means, and how to derive the right one for a new problem. The same principle defines language modelling: Unit 2 trains a model to make the next token of real text as probable as possible.

SKIP TEST

1. Show that maximising the likelihood of a linear model with Gaussian noise of fixed variance is the same as minimising the mean squared error.
2. Why minimise the negative log-likelihood instead of maximising the likelihood itself?
3. What does a naive softmax return for the logits $(1000, 1001, 1002)$ in float32, and how is it fixed?
4. What is the gradient of the cross-entropy loss with respect to the logits?

If all four are easy, read the section on numerical stability and do Labs 1.3.3 and 1.3.4.

Core ideas

Models that assign probabilities

A *probabilistic model* says, for an input x, how probable each possible output y is: a distribution $p(y \mid x; \theta)$ with parameters θ. Given a dataset of n examples (x_i, y_i) drawn independently, the probability the model assigns to all of them together is the product $\prod_i p(y_i \mid x_i; \theta)$, the *likelihood* of θ. *Maximum likelihood estimation* chooses the θ that makes it largest.

Products of thousands of probabilities underflow to zero in floating point, and their derivatives are unwieldy. The logarithm turns the product into a sum and preserves the maximiser, and negating it gives a quantity to minimise. Dividing by n makes it independent of the dataset's size:

$$\mathscr{L}(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \log p(y_i \mid x_i; \theta).$$

This *negative log-likelihood* is the loss. Every model trained in this course, from the regression below to the language models of Unit 2, minimises one.

Squared error and Gaussian noise

Suppose y is a linear function of x plus noise drawn from a normal distribution with standard deviation σ: $y = w^T x + b + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. The density of one observation is

$$p(y \mid x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\left(y - w^T x - b\right)^2}{2\sigma^2}\right),$$

so its negative logarithm is

$$-\log p(y \mid x) = \frac{\left(y - w^T x - b\right)^2}{2\sigma^2} + \log\sigma + \frac{1}{2}\log 2\pi.$$

For fixed σ, only the first term depends on w and b, and minimising its average is minimising the *mean squared error*. The squared error is therefore the right loss when the errors are Gaussian with the same spread everywhere. With heavy-tailed errors it lets a few outliers dominate, and Lab 1.3.1 deals with a case where the spread varies. Minimising over σ as well gives σ^2 equal to the mean squared residual, so the model also learns how noisy the data are.

The program fits 200 synthetic points three ways.

A linear model with Gaussian noise of standard deviation 0.5 — code/u1/regression.py

```
n, true_w, true_b, sigma = 200, np.array([2.0, -1.0, 0.5]), 0.3, 0.5
x = rng.normal(size=(n, 3))
y = (x @ true_w + true_b + rng.normal(scale=sigma, size=n))[:, None]
```

The first solves the least-squares problem directly. Setting the gradient of the squared error to zero gives the *normal equations* $X^T X w = X^T y$; `lstsq` solves them through an orthogonal factorisation of X, which is more accurate than forming and inverting $X^T X$.

Least squares in closed form, with a column of ones for the bias — code/u1/regression.py

```
design = np.hstack([x, np.ones((n, 1))])
solution = np.linalg.lstsq(design, y, rcond=None)[0].ravel()
```

The second minimises the mean squared error with `tiny` and gradient descent; the third minimises the full negative log-likelihood, with $\log\sigma$ as a third parameter so that σ stays positive.

Gradient descent on the mean squared error — code/u1/regression.py

```
w = Tensor(np.zeros((3, 1)), requires_grad=True)
b = Tensor(np.zeros(1), requires_grad=True)
opt = optim.SGD([w, b], lr=0.1)
for _ in range(500):
    residual = Tensor(x) @ w + b - Tensor(y)
    loss = (residual * residual).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
```

Maximum likelihood, learning the noise level too — code/u1/regression.py

```
w2 = Tensor(np.zeros((3, 1)), requires_grad=True)
b2 = Tensor(np.zeros(1), requires_grad=True)
log_sigma = Tensor(np.zeros(1), requires_grad=True)
opt = optim.Adam([w2, b2, log_sigma], lr=0.05)
for _ in range(2000):
    residual = Tensor(x) @ w2 + b2 - Tensor(y)
    var = (log_sigma * 2.0).exp()
    nll = (
        (residual * residual / (var * 2.0)).mean()
        + log_sigma.mean()
        + 0.5 * math.log(2 * math.pi)
    )
    opt.zero_grad()
    nll.backward()
    opt.step()
```

Three routes to the same answer

```
$ uv run -m u1.regression
                              w1      w2      w3       b
true                      2.0000 -1.0000  0.5000  0.3000
least squares             1.9550 -1.0481  0.5265  0.2819
gradient descent on MSE   1.9550 -1.0481  0.5265  0.2819
maximum likelihood        1.9550 -1.0481  0.5265  0.2819
noise: true sigma 0.5, learned sigma 0.50004, RMS residual 0.50004
```

All three agree to four decimal places, and the learned σ equals the root-mean-square residual, as the derivation predicts. The estimates differ from the true parameters by a few hundredths, the sampling error of 200 noisy points; more data would shrink it in proportion to $1/\sqrt{n}$.

Classification: logits, softmax and cross-entropy

For a label that takes one of K values, the natural model is a categorical distribution, K probabilities that sum to one. A network cannot easily output numbers constrained like that, so it outputs K unconstrained scores, *logits* $z_1, \dots, z_K$, and the *softmax* turns them into probabilities:

$$p(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Adding the same constant to every logit leaves the probabilities unchanged; only differences between logits matter. The negative log-likelihood of the true class y is

$$-\log p(y \mid x) = -z_y + \log \sum_j e^{z_j},$$

the *cross-entropy* loss. For $K = 2$ the softmax reduces to the logistic function $\sigma(z) = 1/(1 + e^{-z})$ of the difference of the two logits, and the loss to the binary cross-entropy.

DERIVATION

Differentiate the cross-entropy with respect to the logit z_k. The first term contributes -1 when $k = y$ and nothing otherwise; the second contributes $e^{z_k} / \sum_j e^{z_j} = p_k$. So

$$\frac{\partial \mathscr{L}}{\partial z_k} = p_k - [k = y],$$

the predicted probability minus the one-hot target. The gradient is large where the model is confidently wrong and vanishes as it becomes confidently right. `tiny`'s `log_softmax` uses this result in its backward step.

Softmax regression applies this to images: the logits are a linear function of the 784 pixels, $z = W^T x + b$, with one column of W per digit.

Softmax regression on MNIST, trained with `tiny` — `code/u1/logistic.py`

```
w = Tensor(np.zeros((784, 10), dtype=np.float32), requires_grad=True)
b = Tensor(np.zeros(10, dtype=np.float32), requires_grad=True)
opt = optim.SGD([w, b], lr=0.1)
for epoch in range(1, 6):
    for idx in batches(len(x), 128, rng):
        loss = cross_entropy(Tensor(x[idx]) @ w + b, y[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
    with no_grad():
        val = ((Tensor(x_val) @ w + b).data.argmax(1) == y_val).mean()
    print(f"epoch {epoch}  validation accuracy {val:.2%}")
```

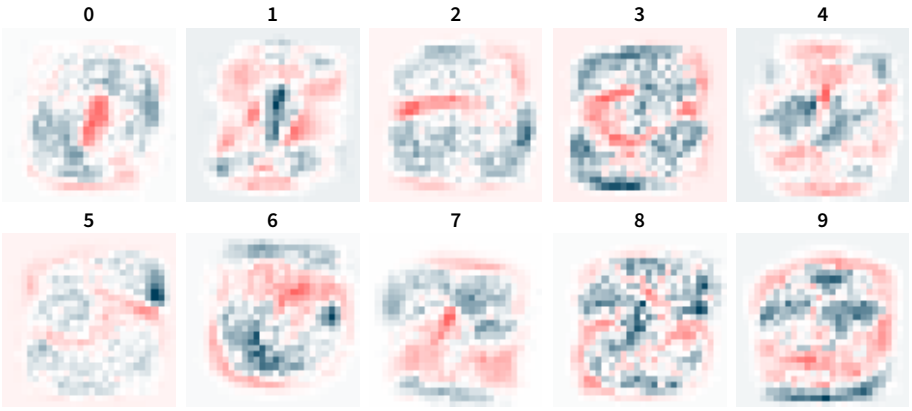


Figure 1.3.1. The ten columns of the trained W as 28×28 images: blue weights favour the digit, red weights count against it.

A linear model reaches 92% on MNIST

```
$ uv run -m u1.logistic
epoch 1  validation accuracy 90.64%
epoch 2  validation accuracy 91.34%
epoch 3  validation accuracy 90.26%
epoch 4  validation accuracy 91.10%
epoch 5  validation accuracy 91.36%
test accuracy 91.87%; most common mistakes: 9 read as 4 (54), 8 read as 5 (51), 5 read as 3
    (37)
```

Each column of W is a template, a pattern of positive and negative weights over the image, and the predicted digit is the template that best matches the input (Figure 1.3.1). The linear model's mistakes show its limit, because a 9 and a 4 differ in a few strokes that a single template per class cannot isolate. Chapter 1.5's networks, which compose many such features, remove most of these errors.

Numerical stability

Floating point has a limited range. `exp` overflows float32 above about 88.7 and float64 above about 709, and logits larger than that are common in trained networks. A naive softmax of $(1000, 1001, 1002)$ divides infinity by infinity:

The softmax as written in the formula — code/u1/softmax.py

```python
def softmax_naive(z: np.ndarray) -> np.ndarray:
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)
```

Because adding a constant to every logit changes nothing, subtracting the largest logit first keeps every exponent at or below zero, so the sum lies between 1 and K and nothing overflows. The logarithm of the sum, needed by the loss, is computed the same way, which is the *log-sum-exp* trick.

The stable softmax, log-sum-exp and cross-entropy — code/u1/softmax.py

```python
def softmax(z: np.ndarray) -> np.ndarray:
    e = np.exp(z - z.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)


def logsumexp(z: np.ndarray) -> np.ndarray:
    m = z.max(axis=-1, keepdims=True)
    return (m + np.log(np.exp(z - m).sum(axis=-1, keepdims=True))).squeeze(-1)


def cross_entropy(z: np.ndarray, target: int) -> float:
    return float(logsumexp(z) - z[target])
```

The naive and stable versions at extreme logits

```
$ uv run -m u1.softmax
naive softmax of [1000, 1001, 1002]:  [nan nan nan]
stable softmax of [1000, 1001, 1002]: [0.09003057 0.24472848 0.66524094]
log of naive softmax of [0, -120]:    [  0. -inf]
log-softmax via logsumexp:            [   0. -120.]
gradient of cross-entropy = softmax - one-hot: largest error 1.5e-10
```

The second pair of lines shows the opposite failure. A probability of e^{-120} underflows to zero in float32, so taking the logarithm of a computed softmax gives $-\infty$, while computing the log-probability directly as $z_k - \log \sum_j e^{z_j}$ gives the exact value, -120. This is why frameworks provide `log_softmax` and a cross-entropy that takes logits, and why a model should never output probabilities to a loss that takes their logarithm.

The softmax-minus-one-hot gradient, checked numerically `code/u1/softmax.py`

```
rng = np.random.default_rng(0)
z = rng.normal(size=5)
target = 2
analytic = softmax(z) - np.eye(5)[target]
h = 1e-6
numeric = np.array(
    [
        (cross_entropy(z + h * e, target) - cross_entropy(z - h * e, target)) / (2 * h)
        for e in np.eye(5)
    ]
)
```

PITFALL

PyTorch's `cross_entropy` takes *logits*, applies `log_softmax` itself, and expects targets as integer class indices or as a probability for every class. Passing probabilities, the output of a softmax, still runs: the loss applies a second softmax, the gradients shrink, and the model trains slowly to a poor result. Chapter 1.7 hides this bug in a training run for you to find.

Units: nats, bits and perplexity

A cross-entropy computed with natural logarithms is in *nats*; divided by $\ln 2$ it is in *bits*, the number of bits an ideal code built from the model's probabilities would need per example. An untrained classifier over ten classes predicts each with probability $1/10$ and scores $\ln 10 \approx 2.30$ nats, which is where an untrained ten-class classifier's loss starts. Language models report the same quantity per token or per byte of text, and its exponential, the *perplexity*: a perplexity of 20 means the model is, on average, as uncertain as a uniform choice among 20 tokens. Unit 2 uses all three.

IN PYTORCH

`torch.nn.functional.cross_entropy(logits, targets)` computes this chapter's stable loss in one call, averaging over the batch; `binary_cross_entropy_with_logits` is its two-class form. `torch.logsumexp` and `torch.log_softmax` are the stable building blocks. The regression's closed form is `torch.linalg.lstsq`.

Reading

- Christopher Bishop and Hugh Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, chapters 4 and 5: single-layer networks for regression and classification.
- Kevin Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, sections 4.2 and 10.2: maximum likelihood and logistic regression.
- Pierre Blanchard, Desmond Higham and Nicholas Higham, "Accurately Computing the Log-Sum-Exp and Softmax Functions", *IMA Journal of Numerical Analysis* 41(4), 2021.

Labs

LAB 1.3.1 When the noise varies — 1.5 hours

Goal. Use maximum likelihood to model noise that varies with the input.

Code. `code/u1/regression.py`

1. Generate data whose noise grows with x: $\sigma(x) = 0.1 + 0.5|x_1|$.
2. Fit a model that predicts both the mean and $\log \sigma(x)$ as linear functions of x, by maximum likelihood in `tiny`.
3. Compare its fitted mean with least squares, and its predicted σ with the truth at five values of x_1.

DONE WHEN

- [ ] Your model recovers $\sigma(x)$ within 20% at all five points.
- [ ] You can explain why least squares gives a reasonable mean but no account of the spread.

EVIDENCE FOR THE NOTEBOOK

- The code, the comparison table, and a plot of the data with the predicted $\pm 2\sigma$ band.

LAB 1.3.2 Logistic and two-class softmax regression — 1 hour

Goal. Confirm that binary logistic regression is softmax regression with two classes.

Code. `code/u1/logistic.py`

1. Restrict MNIST to the digits 3 and 8. Train a logistic regression with one output, the sigmoid and the binary cross-entropy, in `tiny`.
2. Train a softmax regression with two outputs on the same data.
3. Show that the difference of the softmax model's two logits behaves like the logistic model's single logit.

DONE WHEN

- [ ] Both models reach the same test accuracy within half a percentage point.
- [ ] You can state the exact relation between the two parameterisations.

EVIDENCE FOR THE NOTEBOOK

- Both accuracies and your derivation of the relation.

LAB 1.3.3 A stable binary cross-entropy — 1 hour

Goal. Write losses that are exact at any logit.

Code. `code/u1/softmax.py`

1. Write a binary cross-entropy from a logit, `bce(z, y)`, that is exact for z from -1000 to 1000, using $\log\left(1 + e^{-|z|}\right)$.
2. Test it in float32 and float64 against a high-precision reference from Python's `decimal` module.
3. Add a gradient check to show that its derivative is $\sigma(z) - y$.

DONE WHEN

- [ ] Your function matches the reference to within a relative error of 10^{-6} in float32 at every test point.
- [ ] The gradient check passes.

EVIDENCE FOR THE NOTEBOOK

- The function, the error table and the check.

LAB 1.3.4 Reading the weights — 1 hour

Goal. Interpret a linear classifier and its mistakes.

Code. `code/u1/logistic.py`

1. Draw the ten weight templates from your own run.
2. For the three most common confusions, show two misclassified test images each, with the model's top two probabilities.
3. Add the squared value of every pixel as 784 extra features and retrain. Report the accuracy and explain what the extra features can and cannot express.

DONE WHEN

- [ ] You can explain each confusion from the templates.
- [ ] You have the accuracy with the extra features, and a reason for the change.

EVIDENCE FOR THE NOTEBOOK

- The images, the probabilities and the explanation.

Problem set

1. Derive the loss that maximum likelihood gives when the noise follows a Laplace distribution, $p(\varepsilon) \propto e^{-|\varepsilon|/b}$. Which statistic of the residuals does it make the bias estimate?
2. Show that the softmax is unchanged by adding a constant to every logit, and that it is changed by multiplying them by a constant. What does the multiplier do to the distribution?
3. A classifier outputs the probability 0.9 for the correct class on every example. What is its cross-entropy in nats and in bits? What accuracy might it have?
4. Why is $\log(1 + e^z)$ computed as $\max(z, 0) + \log\left(1 + e^{-|z|}\right)$? Which goes wrong in float32 for $z = 100$ and for $z = -100$ in the naive form?
5. The normal equations can be solved by inverting $X^T X$. Explain, with the condition number, why `lstsq` is preferred, and give a dataset on which inversion fails.

COMPLETION CHECKLIST

- [ ] I can derive the mean squared error and the cross-entropy from maximum likelihood, and say what each assumes.
- [ ] I can explain logits, the softmax and the gradient of the cross-entropy.
- [ ] I can compute softmax, log-softmax and cross-entropy stably, and explain why the naive forms fail.
- [ ] I can convert a loss between nats, bits and perplexity.
- [ ] Lab 1.3.1–1.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.4

Optimisers

TIME	5 hours
YOU BUILD	SGD, momentum, Nesterov momentum, RMSProp, Adam and AdamW written from their update rules and matched to PyTorch's to sixteen digits; their paths through an ill-conditioned valley; a learning-rate range test; and the warm-up and cosine schedule that Unit 2's pretraining uses.
NEEDS	Chapters 1.2 and 1.3.

WHY IT MATTERS

The gradient gives the downhill direction but not how far to step, or how to use what earlier steps showed about the shape of the loss. The optimiser handles both. The learning rate is the setting that most often decides whether a training run succeeds. If it is too small, the loss barely falls for hours, and if it is too large, the loss diverges. This chapter derives the optimisers in common use from what goes wrong with plain gradient descent, implements each and checks that it matches PyTorch's, and ends with methods for choosing a learning rate by measurement.

SKIP TEST

1. Gradient descent minimises $f(x) = \frac{1}{2}\lambda x^2$ with step size η. For which η does it converge, and for which is convergence fastest?
2. What does momentum do in a long, narrow valley, and roughly how many fewer steps can it take?
3. What is Adam's bias correction for, and what would happen in the first steps without it?
4. How does AdamW's weight decay differ from adding an L2 penalty to the loss when using Adam?

If all four are easy, read the sections on Adam and schedules, and do Labs 1.4.3 and 1.4.4.

Core ideas

Gradient descent and its step size

Gradient descent repeats $\theta \leftarrow \theta - \eta \nabla \mathscr{L}(\theta)$ with a *learning rate* η. Its behaviour is easiest to see on a quadratic, which every smooth loss resembles near a minimum. On $f(x) = \frac{1}{2}\lambda x^2$ the update is $x \leftarrow (1 - \eta\lambda)x$, so the error shrinks by the factor $|1 - \eta\lambda|$ each step. Descent therefore converges if $0 < \eta < 2/\lambda$, fastest at $\eta = 1/\lambda$, and oscillates with growing amplitude beyond $2/\lambda$.

A real loss curves differently in different directions. Near a minimum it is approximately $\frac{1}{2}(\theta - \theta^*)^T H(\theta - \theta^*)$, where the *Hessian* H holds the second derivatives, and each eigenvector of H is a direction with its own curvature λ_i. A single learning rate must serve them all. It must stay below $2/\lambda_{\max}$ to avoid diverging along the sharpest direction, and then progress along the flattest takes about $\lambda_{\max}/\lambda_{\min}$ steps to shrink the error by a constant factor. That ratio, the *condition number* κ, measures how hard a problem is for gradient descent, and a long, narrow valley has a large one.

A quadratic valley with curvatures 1 and 50, rotated by 30 degrees — code/u1/valley.py

```
ANGLE = math.radians(30)
R = np.array([[math.cos(ANGLE), -math.sin(ANGLE)], [math.sin(ANGLE), math.cos(ANGLE)]])
H = R @ np.diag([1.0, 50.0]) @ R.T


def loss(v: Tensor) -> Tensor:
    return 0.5 * (v @ Tensor(H) * v).sum()
```

In practice the gradient is also noisy. Training computes it on a *minibatch* of examples rather than the whole dataset, which makes each step far cheaper and adds noise whose size shrinks as the batch grows. This *stochastic gradient descent* is what "SGD" means. The noise also helps the parameters escape sharp minima, which tend to generalise worse.

Momentum

In a narrow valley, gradient descent zig-zags across the steep walls while creeping along the floor. *Momentum* keeps a running sum of past gradients, a velocity, and steps along it:

$$b \leftarrow \mu b + g, \quad \theta \leftarrow \theta - \eta b.$$

Components of the gradient that keep their sign, along the valley floor, accumulate, while components that alternate, across it, cancel. With μ and η tuned to the curvatures, the number of steps falls from about κ to about $\sqrt{\kappa}$. *Nesterov momentum* evaluates the step as if the velocity had already been applied, which damps overshooting. In the form PyTorch uses, it steps along $g + \mu b$ instead of b.

SGD with optional momentum, Nesterov momentum and weight decay, as PyTorch defines them — code/aif/tiny/optim.py

```
class SGD(Optimizer):
    def __init__(
        self,
        params,
        lr: float,
        momentum: float = 0.0,
        nesterov: bool = False,
        weight_decay: float = 0.0,
    ):
        super().__init__(params, lr)
        self.momentum, self.nesterov, self.weight_decay = momentum, nesterov, weight_decay

    def update(self, p, g, state):
        if self.weight_decay:
            g = g + self.weight_decay * p.data
        if self.momentum:
            buf = state.get("buf")
            buf = g.copy() if buf is None else self.momentum * buf + g
            state["buf"] = buf
            g = g + self.momentum * buf if self.nesterov else buf
        p.data -= self.lr * g
```

Adaptive methods: RMSProp and Adam

Momentum still uses one learning rate for every parameter. In a network, gradients for different parameters can differ in size by orders of magnitude, for example between an embedding of a rare word and the bias of a busy layer. *RMSProp* divides each parameter's step by a running root-mean-square of its own gradients, so each moves at a similar rate:

$$v \leftarrow \alpha v + (1-\alpha) g^2, \quad \theta \leftarrow \theta - \eta g / \left(\sqrt{v} + \varepsilon\right).$$

Adam combines this with momentum. It keeps exponential averages of the gradient, m, and of its square, v, and steps along $m/\sqrt{v}$. Both averages start at zero, so in the first steps they underestimate their targets: after one step $m = (1-\beta_1)g$, a tenth of g with the usual $\beta_1 = 0.9$. Dividing by $1-\beta^t$ corrects this exactly for a constant gradient, and its effect fades as t grows.

Adam with bias correction, and AdamW's decoupled weight decay — code/aif/tiny/optim.py

```python
class Adam(Optimizer):
    def __init__(
        self,
        params,
        lr: float,
        betas=(0.9, 0.999),
        eps: float = 1e-8,
        weight_decay: float = 0.0,
        decoupled: bool = False,
    ):
        super().__init__(params, lr)
        self.beta1, self.beta2 = betas
        self.eps, self.weight_decay, self.decoupled = eps, weight_decay, decoupled

    def update(self, p, g, state):
        if self.weight_decay and self.decoupled:
            p.data -= self.lr * self.weight_decay * p.data
        elif self.weight_decay:
            g = g + self.weight_decay * p.data
        t = state["t"] = state.get("t", 0) + 1
        m = state["m"] = self.beta1 * state.get("m", 0.0) + (1 - self.beta1) * g
        v = state["v"] = self.beta2 * state.get("v", 0.0) + (1 - self.beta2) * g * g
        m_hat = m / (1 - self.beta1**t)
        v_hat = v / (1 - self.beta2**t)
        p.data -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)


class AdamW(Adam):
    def __init__(
        self,
        params,
        lr: float,
        betas=(0.9, 0.999),
        eps: float = 1e-8,
        weight_decay: float = 0.01,
    ):
        super().__init__(params, lr, betas, eps, weight_decay, decoupled=True)
```

Weight decay shrinks parameters towards zero each step, a regulariser that Chapter 1.6 studies. For SGD, decaying the weights is the same as adding $\frac{1}{2}\lambda\|\theta\|^2$ to the loss. For Adam the two differ, because a penalty's gradient passes through the division by $\sqrt{v}$, so parameters with large gradients are barely regularised. *AdamW* applies the decay to the weights directly, outside the adaptive step, and is the default for training transformers.

RMSProp — code/aif/tiny/optim.py

```python
class RMSProp(Optimizer):
    def __init__(self, params, lr: float, alpha: float = 0.99, eps: float = 1e-8):
        super().__init__(params, lr)
        self.alpha, self.eps = alpha, eps

    def update(self, p, g, state):
        v = state.get("v", np.zeros_like(p.data))
        v = self.alpha * v + (1 - self.alpha) * g * g
        state["v"] = v
        p.data -= self.lr * g / (np.sqrt(v) + self.eps)
```

Each optimiser is checked against PyTorch's on the same problem, from the same starting point, for fifty steps:

Six pairs of optimisers, tiny's and PyTorch's — code/u1/optim_check.py

```
pairs = {
    "SGD": (lambda p: optim.SGD(p, 0.1), lambda p: torch.optim.SGD(p, 0.1)),
    "SGD, momentum 0.9": (
        lambda p: optim.SGD(p, 0.1, momentum=0.9),
        lambda p: torch.optim.SGD(p, 0.1, momentum=0.9),
    ),
    "SGD, Nesterov": (
        lambda p: optim.SGD(p, 0.1, momentum=0.9, nesterov=True),
        lambda p: torch.optim.SGD(p, 0.1, momentum=0.9, nesterov=True),
    ),
    "RMSProp": (lambda p: optim.RMSProp(p, 0.01), lambda p: torch.optim.RMSprop(p, 0.01)),
    "Adam, L2 0.1": (
        lambda p: optim.Adam(p, 0.01, weight_decay=0.1),
        lambda p: torch.optim.Adam(p, 0.01, weight_decay=0.1),
    ),
    "AdamW, decay 0.1": (
        lambda p: optim.AdamW(p, 0.01, weight_decay=0.1),
        lambda p: torch.optim.AdamW(p, 0.01, weight_decay=0.1),
    ),
}
```

Identical to within float64 rounding

```
$ uv run -m u1.optim_check
optimiser              max |w_tiny - w_torch| after 50 steps
SGD                                                 5.6e-17
SGD, momentum 0.9                                   2.0e-16
SGD, Nesterov                                       8.9e-17
RMSProp                                             2.8e-16
Adam, L2 0.1                                        8.3e-17
AdamW, decay 0.1                                    7.3e-17
```

Paths through a valley

The valley above has condition number 50. The program runs four optimisers from the same point until the loss falls below 10^{-8}, with gradient descent and momentum at the step sizes that theory says are best for this valley:

Optimal settings for gradient descent and the heavy ball, from the two curvatures — code/u1/valley.py

```
gd_lr = 2 / (lam_min + lam_max)
hb_lr = 4 / (math.sqrt(lam_max) + math.sqrt(lam_min)) ** 2
hb_mu = ((math.sqrt(kappa) - 1) / (math.sqrt(kappa) + 1)) ** 2
contenders = {
    "gradient descent, lr 0.039": lambda p: optim.SGD(p, gd_lr),
    f"momentum, lr {hb_lr:.3f}, mu {hb_mu:.2f}": lambda p: optim.SGD(
        p, hb_lr, momentum=hb_mu
    ),
    "RMSProp, lr 0.01": lambda p: optim.RMSProp(p, 0.01),
    "Adam, lr 0.1": lambda p: optim.Adam(p, 0.1),
}
```

Steps to converge on the rotated valley

```
$ uv run -m u1.valley
quadratic valley with condition number 50, rotated 30 degrees
  gradient descent, lr 0.039       272 steps to loss < 1e-8
  momentum, lr 0.061, mu 0.57       54 steps to loss < 1e-8
  RMSProp, lr 0.01                 877 steps to loss < 1e-8
  Adam, lr 0.1                     294 steps to loss < 1e-8
```

Momentum needs 54 steps to gradient descent's 272, five times fewer, of the order of the factor $\sqrt{\kappa} \approx 7$ that theory predicts. The adaptive methods gain nothing here, because their per-parameter scaling is aligned with the coordinate axes and this valley is rotated, so the scaling does not undo its shape. With a constant learning rate they also settle slowly near the minimum. Adam needs a few more steps than gradient

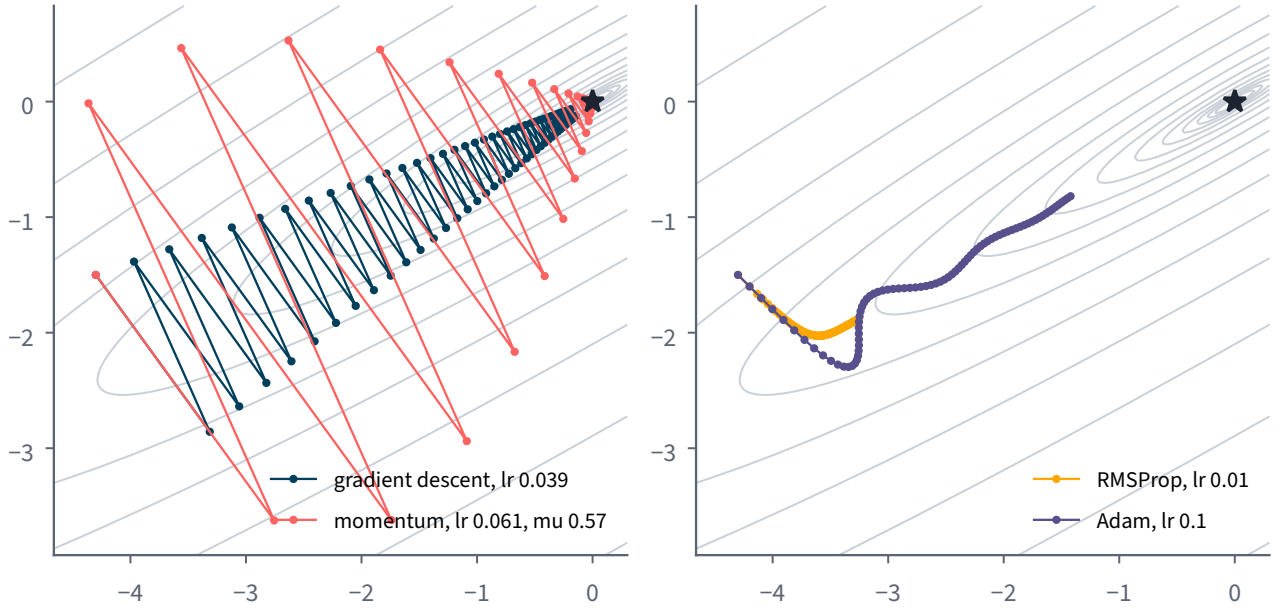


Figure 1.4.1. The first 80 steps of each optimiser on the valley's contours, from the same starting point at the lower left. Left: gradient descent zig-zags across the valley; momentum swings further but travels faster along it. Right: RMSProp and Adam set off along the diagonal of the coordinate axes before turning down the valley.

descent, and RMSProp, which has no momentum, three times as many. On neural networks the balance is reversed, because the curvature there does vary along the parameters themselves, the gradients are noisy, and nobody can compute the optimal momentum. This experiment shows only that no optimiser is best everywhere, and that the learning rate matters to all of them.

> KEY IDEA
> An optimiser's job is to make progress along flat directions without diverging along steep ones. Momentum does it by averaging over time, adaptive methods by rescaling each parameter, and Adam does both. All of them still need a learning rate that suits the problem.

Schedules: warm-up and decay

A constant learning rate is rarely best. Early in training, the parameters are random, the gradients large and erratic, and Adam's averages unreliable. A small learning rate that rises over the first few hundred steps, a *warm-up*, avoids early divergence. Late in training, a smaller learning rate lets the parameters settle into the bottom of the basin instead of bouncing around it. The schedule used throughout this course rises linearly during warm-up and then follows half a cosine down to a tenth of its peak:

Linear warm-up, then cosine decay to a floor — code/aif/train.py

```
def lr_at(step: int, total: int, peak: float, warmup: int, floor: float = 0.1) -> float:
    if step < warmup:
        return peak * (step + 1) / warmup
    progress = min(1.0, (step - warmup) / max(1, total - warmup))
    return peak * (floor + (1 - floor) * 0.5 * (1 + math.cos(math.pi * progress)))
```

Finding a learning rate

A *learning-rate range test* trains for a few hundred steps while raising the learning rate exponentially from tiny to huge, and records the smoothed loss. The loss first barely moves, then falls quickly, then turns up and diverges.

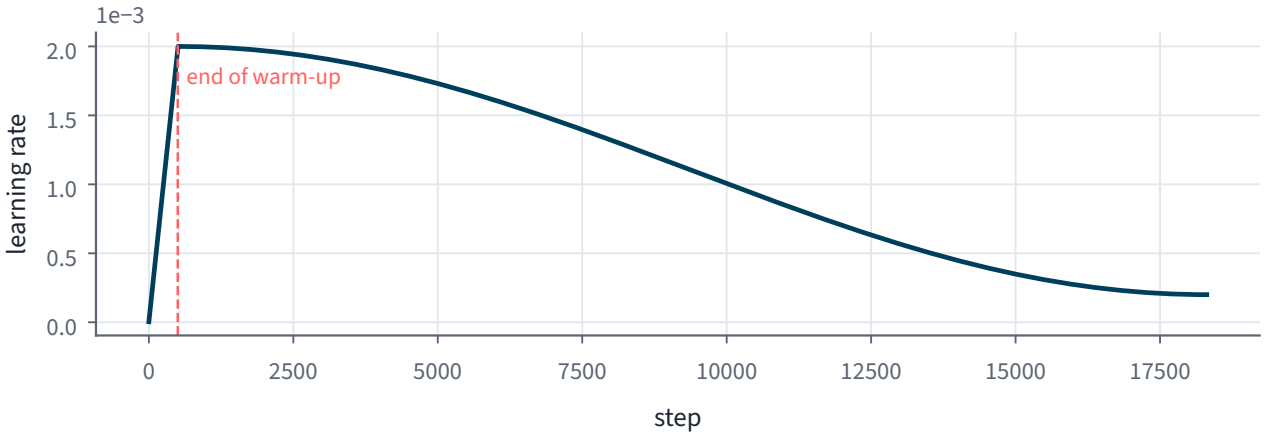


Figure 1.4.2. The schedule of Unit 2's pretraining run: 500 steps of warm-up to 2×10^{-3}, then cosine decay to a tenth of the peak over the remaining 17,810 steps.

A range test: one short run with an exponentially rising learning rate — code/u1/lr_range.py

```
low, high, steps = 1e-6, 1.0, 300
record, smooth = [], None
order = (idx for _ in range(10) for idx in batches(len(x), 128, rng))
for step in range(steps):
    opt.lr = low * (high / low) ** (step / (steps - 1))
    idx = next(order)
    loss = cross_entropy(model(Tensor(x[idx])), y[idx])
    opt.zero_grad()
    loss.backward()
    opt.step()
    smooth = loss.item() if smooth is None else 0.9 * smooth + 0.1 * loss.item()
    record.append((opt.lr, smooth))
    if not math.isfinite(smooth) or smooth > 1.5 * record[0][1]:
        break
```

The range test for Chapter 1.5's network with AdamW

```
$ uv run -m u1.lr_range
lowest smoothed loss 0.387 at lr 7.8e-03; loss back above its start at lr 1.0e-01; stopped
    at lr 2.2e-01
suggested learning rate: about 8e-04, a tenth of the minimum's
```

The test conflates two things, since later steps have both a higher rate and more training behind them, so it indicates only a range of suitable rates. A tenth of the rate at the minimum is a sound starting point, and

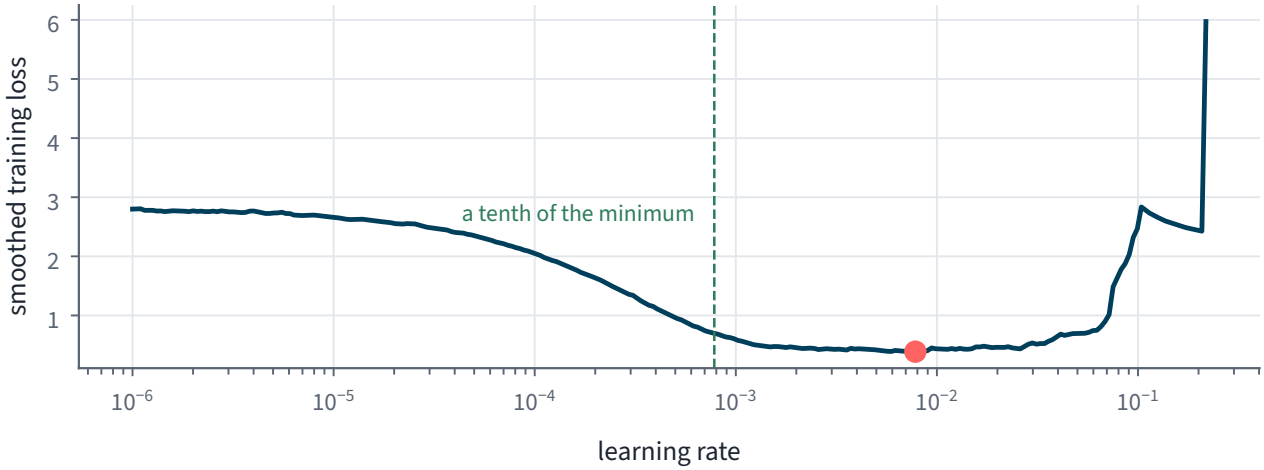


Figure 1.4.3. Smoothed loss against learning rate. Training is fastest just before the loss turns up; a tenth of that rate is a safe choice for a full run.

is close to the 10^{-3} that Chapter 1.5 uses. For expensive runs a short sweep is better still: Unit 2 chose its pretraining rate by three runs of six million tokens each.

> **IN PYTORCH**
> `torch.optim` provides every optimiser here, with `param_groups` for settings that differ between parameters, such as no weight decay for biases and normalisation weights. Schedules live in `torch.optim.lr_scheduler`, or are written as this course writes them, setting `group["lr"]` each step. `torch.nn.utils.clip_grad_norm_` rescales the gradient when its norm exceeds a limit, a guard against the occasional huge step.

Reading

- Gabriel Goh, "Why Momentum Really Works", *Distill*, 2017.
- Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.
- Ilya Loshchilov and Frank Hutter, "Decoupled Weight Decay Regularization", ICLR 2019.
- Leslie Smith, "Cyclical Learning Rates for Training Neural Networks", WACV 2017, section 3.3: the range test.

Labs

LAB 1.4.1 The divergence threshold of gradient descent — 1 hour

Goal. Find the divergence threshold of gradient descent and compare it with theory.

Code. `code/u1/valley.py`

1. On the valley, run gradient descent at step sizes from 0.01 to 0.05 and record the final loss after 200 steps.
2. Locate the step size at which it diverges and compare it with $2/\lambda_{\max}$.
3. Repeat with the curvature 50 changed to 200 and note how the best step size and the steps to converge change.

DONE WHEN

- [ ] Your measured threshold agrees with $2/\lambda_{\max}$ to within 2%.
- [ ] You can state how the steps to converge scale with the condition number, with your measurements.

EVIDENCE FOR THE NOTEBOOK

- The table of final losses and the threshold for both valleys.

LAB 1.4.2 Tuning momentum — 1.5 hours

Goal. Map how steps to converge depend on μ and η.

Code. `code/u1/valley.py`

1. Run heavy-ball momentum on a grid of μ from 0 to 0.95 and η from 0.01 to 0.1, recording steps to reach a loss below 10^{-8}.
2. Mark the theoretical optimum on your grid.
3. Repeat with Nesterov momentum and compare the size of the good region.

DONE WHEN

- [ ] Your heat map shows the region where momentum beats gradient descent's 272 steps.
- [ ] You can say which of the two methods is less sensitive to its settings.

EVIDENCE FOR THE NOTEBOOK
- Both heat maps and a paragraph comparing them.

LAB 1.4.3 SGD against Adam on MNIST — 1.5 hours

Goal. Compare the two optimisers on a neural network.

Code. `code/u1/mnist_tiny.py`

1. Train Chapter 1.5's network for three epochs with SGD at learning rates 0.01, 0.03 and 0.1, momentum 0.9, and with AdamW at 3×10^{-4}, 10^{-3} and 3×10^{-3}.
2. Plot the six training-loss curves on one chart, and record the validation accuracy of each.

DONE WHEN
- [ ] You have six curves and six accuracies.
- [ ] You can say which optimiser was more forgiving of a badly chosen learning rate, and support it from the chart.

EVIDENCE FOR THE NOTEBOOK
- The chart, the accuracies and the conclusion.

LAB 1.4.4 Range test and schedule — 1 hour

Goal. Choose a learning rate by measurement and test a schedule.

Code. `code/u1/lr_range.py`

1. Run the range test three times with different seeds, `--seed 1`, `2` and `3`, and record the rate at the minimum each time.
2. Train the network for eight epochs at the suggested rate with a constant rate, and with warm-up over one epoch and cosine decay.

DONE WHEN
- [ ] Your three range tests agree within a factor of two.
- [ ] You have the final validation accuracy of both runs and can say whether the schedule helped.

EVIDENCE FOR THE NOTEBOOK
- The three range-test plots and the two learning curves.

Problem set

1. Show that gradient descent on $f(x) = \frac{1}{2}\lambda x^2$ converges in one step at $\eta = 1/\lambda$. Why is this step size not available in a real network?
2. After t steps with a constant gradient g, what are Adam's m and v before and after bias correction? What step does it take?
3. Adam's step is roughly η in size for every parameter whatever its gradient's scale. Why does that make Adam's learning rate easier to transfer between models than SGD's?
4. A training run with batch size 32 works at learning rate 10^{-3}. You raise the batch size to 512. Give two rules of thumb for the new learning rate and say when each applies.
5. With weight decay λ and learning rate η, by what factor does AdamW shrink a weight per step in the absence of gradient? Over 18,000 steps at $\eta = 2 \times 10^{-3}$ and $\lambda = 0.1$?

COMPLETION CHECKLIST

- [ ] I can explain the step-size limit and the role of the condition number for gradient descent.
- [ ] I can derive momentum, RMSProp and Adam from the problems each one addresses, including bias correction.
- [ ] I can explain the difference between L2 regularisation and decoupled weight decay.
- [ ] I can choose a learning rate with a range test or a short sweep, and explain warm-up and decay.
- [ ] Lab 1.4.1–1.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.5

Neural networks

TIME	6 hours
YOU BUILD	Activation scales measured through thirty layers for five initialisations, with He initialisation derived from the variance of a layer; an MNIST classifier trained to 97.9% in your own library; and its PyTorch twin, matched to `tiny` step for step for a hundred steps, then timed on the CPU and the GPU.
NEEDS	Chapters 1.2 to 1.4.

WHY IT MATTERS

A linear model stops at 92% on MNIST, because one template per digit cannot capture the ways people write it. Composing simple functions, each a linear map followed by a nonlinearity, gives models that can represent far richer patterns, and that learn features no one designed. This chapter builds such networks in `tiny`, explains why the scale of their initial weights decides whether they can be trained at all, and puts the hand-built library side by side with PyTorch.

SKIP TEST

1. Why is a stack of linear layers with no activation functions between them no more expressive than one linear layer?
2. A layer has n inputs of unit variance and a ReLU. What variance must its weights have to keep its outputs' second moment equal to its inputs'?
3. What happens to the activations of a 30-layer ReLU network whose weights are drawn with standard deviation 0.01?
4. Why can a small network train faster on a CPU than on a GPU?

If all four are easy, read the section on initialisation and do Labs 1.5.1 and 1.5.3.

Core ideas

Layers and nonlinearities

A *multilayer perceptron* (MLP) alternates linear maps and element-wise nonlinear functions:

$$h_1 = \varphi\big(W_1^T x + b_1\big), \quad h_2 = \varphi\big(W_2^T h_1 + b_2\big), \quad \ldots, \quad z = W_L^T h_{L-1} + b_L.$$

The vectors h_l are *hidden activations*, and the last layer outputs the logits of Chapter 1.3. Without the nonlinearity φ the composition would collapse into a single linear map, since a product of matrices is a matrix. With it, each layer can carve its input space along new boundaries, and a network with one wide enough hidden layer can approximate any continuous function on a bounded region as closely as desired. Depth makes this efficient, because many functions that a deep network represents compactly would need exponentially many units in a shallow one.

The usual nonlinearity is the *rectified linear unit*, $\mathrm{ReLU}(z) = \max(0, z)$. It is cheap, and its gradient is 1 wherever it is active, so gradients pass through it undiminished. `tanh` and the sigmoid saturate for large inputs, where their gradients vanish. Transformers use smooth relatives of ReLU, such as GELU and SiLU, which Unit 2 meets.

In `tiny`, a network is a tree of *modules*. A module holds parameters and sub-modules, and finds its parameters by walking its attributes:

The base class: parameters, a training flag, and calling `forward` code/aif/tiny/nn.py

```
class Module:
    training = True

    def parameters(self) -> list[Tensor]:
        found = []
        for value in vars(self).values():
            items = value if isinstance(value, (list, tuple)) else [value]
            for item in items:
                if isinstance(item, Tensor) and item.requires_grad:
                    found.append(item)
                elif isinstance(item, Module):
                    found.extend(item.parameters())
        return found

    def modules(self) -> list["Module"]:
        found = [self]
        for value in vars(self).values():
            for item in value if isinstance(value, (list, tuple)) else [value]:
                if isinstance(item, Module):
                    found.extend(item.modules())
        return found

    def train(self, mode: bool = True) -> "Module":
        for m in self.modules():
            m.training = mode
        return self

    def eval(self) -> "Module":
        return self.train(False)

    def zero_grad(self) -> None:
        for p in self.parameters():
            p.grad = None

    def __call__(self, *args):
        return self.forward(*args)
```

A linear layer with weights of shape (inputs, outputs), initialised with a gain code/aif/tiny/nn.py

```
class Linear(Module):
    def __init__(
        self,
        n_in: int,
        n_out: int,
        rng: np.random.Generator,
        gain: float = 2.0,
        dtype=np.float32,
    ):
        std = np.sqrt(gain / n_in)
        self.weight = Tensor(
            rng.normal(0.0, std, (n_in, n_out)).astype(dtype), requires_grad=True
        )
        self.bias = Tensor(np.zeros(n_out, dtype=dtype), requires_grad=True)

    def forward(self, x: Tensor) -> Tensor:
        return x @ self.weight + self.bias
```

An MLP: linear layers with ReLU between them and none after the last — code/aif/tiny/nn.py

```
def mlp(sizes: list[int], rng: np.random.Generator, dropout: float = 0.0) -> Sequential:
    layers = []
    for i, (n_in, n_out) in enumerate(zip(sizes, sizes[1:])):
        last = i == len(sizes) - 2
        layers.append(Linear(n_in, n_out, rng, gain=1.0 if last else 2.0))
        if not last:
            layers.append(ReLU())
            if dropout:
                layers.append(Dropout(dropout, rng))
    return Sequential(*layers)
```

Initialisation

Before any training, a network's output depends only on its random initial weights, and their scale decides how signals travel through it. Consider a layer $z = W^T h$ with n inputs, whose weights are independent, of mean zero and variance $\mathrm{Var}(w)$. Each output is a sum of n products, so

$$\mathrm{Var}(z) = n\,\mathrm{Var}(w) E\left[h^2\right].$$

If h comes from a ReLU applied to a symmetric z' of the previous layer, half of z' is cut to zero and $E\left[h^2\right] = \frac{1}{2}\mathrm{Var}(z')$. Keeping the variance constant from layer to layer therefore needs

$$\mathrm{Var}(w) = \frac{2}{n},$$

He initialisation. For `tanh`, which is close to the identity near zero, the factor 2 disappears: $\mathrm{Var}(w) = 1/n$, *Xavier initialisation*, in its simplest form. Any other scale multiplies the variance by a constant factor at every layer, so after L layers it has grown or shrunk by that factor to the power L.

Push a batch through thirty random layers and record the scale of each layer's output — code/u1/init.py

```
def activation_scales(
    std_fn, activation, depth: int = 30, width: int = 512, seed: int = 0
) -> list[float]:
    rng = np.random.default_rng(seed)
    h = rng.normal(size=(1000, width))
    scales = []
    for _ in range(depth):
        w = rng.normal(0.0, std_fn(width), (width, width))
        h = activation(h @ w)
        scales.append(float(h.std()))
    return scales
```

Five choices of weight scale — code/u1/init.py

```
schemes = {
    "std 0.01, ReLU": (lambda n: 0.01, relu),
    "std 0.1, ReLU": (lambda n: 0.1, relu),
    "Xavier 1/sqrt(n), ReLU": (lambda n: np.sqrt(1 / n), relu),
    "He sqrt(2/n), ReLU": (lambda n: np.sqrt(2 / n), relu),
    "Xavier 1/sqrt(n), tanh": (lambda n: np.sqrt(1 / n), np.tanh),
}
```

Activation scale after 1, 10 and 30 layers of width 512

```
$ uv run -m u1.init
initialisation                layer 1    layer 10    layer 30
std 0.01, ReLU                  0.132    7.97e-09    8.24e-25
std 0.1, ReLU                    1.32        79.7    8.24e+05
Xavier 1/sqrt(n), ReLU          0.585      0.0227    1.89e-05
He sqrt(2/n), ReLU              0.827       0.725        0.62
Xavier 1/sqrt(n), tanh          0.629       0.224       0.127
```

With a standard deviation of 0.01 the signal shrinks by a factor of about 6.3 per layer, to about 8×10^{-25} by the thirtieth. With 0.1 it grows, reaching 10^6. Xavier's scale, right for `tanh`, loses half the variance at every ReLU. The gradients flowing backwards obey the same arithmetic, so a network whose activations vanish

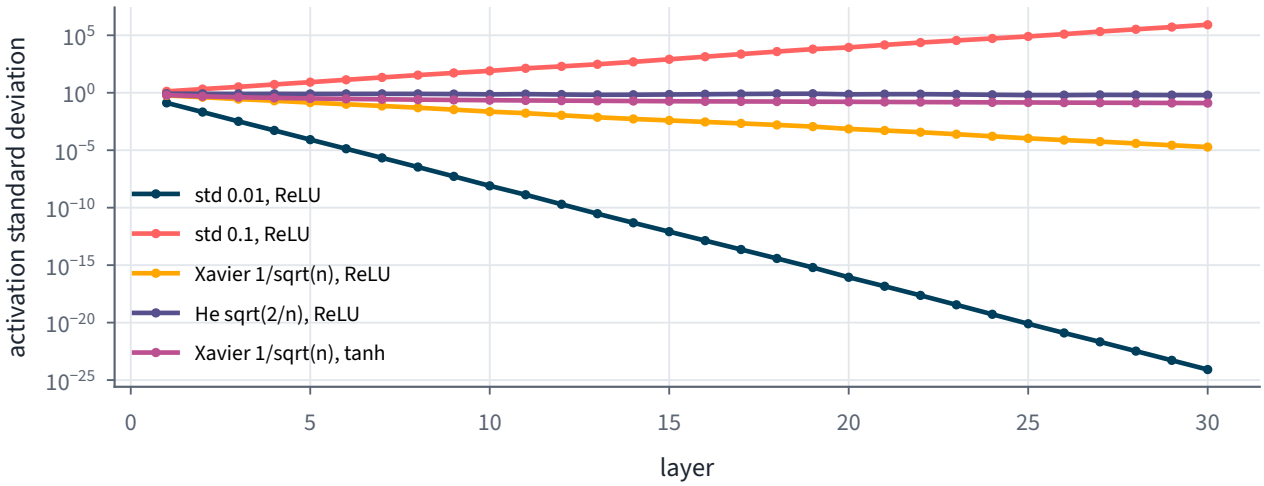


Figure 1.5.1. The standard deviation of each layer's activations, on a logarithmic scale. Only He initialisation holds it steady through thirty ReLU layers.

also has vanishing gradients, and cannot be trained. `tiny`'s `Linear` uses He's scale by default, and Xavier's for the last layer, which has no ReLU after it.

Training on MNIST

MNIST's images are scaled to $[0, 1]$ and then standardised with the training set's mean and standard deviation, so that inputs, like activations, have roughly unit scale. Five thousand training images are held back as a *validation set*, for choosing settings and watching for overfitting; the 10,000 test images are touched only at the end.

Standardised images and a fixed validation split — code/u1/common.py

```
def mnist_split(seed: int = 0, dtype=np.float32, validation: int = 5000):
    d = data.mnist()

    def prepare(images):
        x = images.reshape(len(images), -1).astype(dtype) / 255
        return (x - dtype(0.1307)) / dtype(0.3081)

    x, y = prepare(d["x_train"]), d["y_train"]
    order = np.random.default_rng(seed).permutation(len(x))
    val, train = order[:validation], order[validation:]
    return (x[train], y[train]), (x[val], y[val]), (prepare(d["x_test"]), d["y_test"])
```

The network has two hidden layers of 256 and 128 units, 235,146 parameters, and trains with AdamW at the learning rate found by Chapter 1.4's range test:

The training loop: shuffle, forward, loss, backward, step — code/u1/mnist_tiny.py

```
model = nn.mlp([784, 256, 128, 10], rng)
opt = optim.AdamW(model.parameters(), lr=args.lr, weight_decay=0.01)
for epoch in range(1, args.epochs + 1):
    start, total, count = time.perf_counter(), 0.0, 0
    for idx in batches(len(x), args.batch, rng):
        loss = cross_entropy(model(Tensor(x[idx])), y[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
        total, count = total + loss.item() * len(idx), count + len(idx)
    print(
        f"epoch {epoch}  loss {total / count:.4f}  validation accuracy "
        f"{accuracy(model, x_val, y_val):.2%}  {time.perf_counter() - start:.1f} s"
    )
```

Eight epochs of `tiny` on the M2 Pro's CPU

```
$ uv run -m u1.mnist_tiny
epoch 1  loss 0.2459  validation accuracy 94.96%  0.4 s
epoch 2  loss 0.0958  validation accuracy 96.98%  0.4 s
epoch 3  loss 0.0643  validation accuracy 96.74%  0.4 s
epoch 4  loss 0.0446  validation accuracy 97.30%  0.4 s
epoch 5  loss 0.0368  validation accuracy 97.02%  0.4 s
epoch 6  loss 0.0293  validation accuracy 97.28%  0.4 s
epoch 7  loss 0.0242  validation accuracy 97.54%  0.4 s
epoch 8  loss 0.0186  validation accuracy 97.38%  0.4 s
test accuracy 97.89% with 235,146 parameters
```

The test accuracy of 97.9% cuts the linear model's error rate to about a quarter. Validation accuracy varies from epoch to epoch by a few tenths of a percentage point, partly from the noise of a 5,000-image estimate and partly from the training itself. The test score is measured once, at the end.

The same network in PyTorch

PyTorch computes the same things with the same arithmetic, so a twin of the network, built from `tiny`'s initial weights, should follow it exactly. PyTorch's `Linear` stores its weight as (outputs, inputs), the transpose of `tiny`'s:

Copying `tiny`'s weights into a PyTorch network — code/u1/mnist_torch.py

```
def torch_twin(model: nn.Sequential, dtype=torch.float64) → torch.nn.Sequential:
    layers = []
    for layer in model.layers:
        if isinstance(layer, nn.Linear):
            linear = torch.nn.Linear(*layer.weight.shape, dtype=dtype)
            with torch.no_grad():
                linear.weight.copy_(torch.tensor(layer.weight.data.T))
                linear.bias.copy_(torch.tensor(layer.bias.data))
            layers.append(linear)
        else:
            layers.append(torch.nn.ReLU())
    return torch.nn.Sequential(*layers)
```

Both networks trained on identical batches, in float64 — code/u1/mnist_torch.py

```
for _, idx in zip(range(steps), batches(len(x), 128, rng)):
    xb, yb = x[idx].astype(np.float64), y[idx]
    loss = cross_entropy(mine(Tensor(xb)), yb)
    opt_mine.zero_grad()
    loss.backward()
    opt_mine.step()
    ref = F.cross_entropy(theirs(torch.tensor(xb)), torch.tensor(yb))
    opt_theirs.zero_grad()
    ref.backward()
    opt_theirs.step()
    worst = max(worst, abs(loss.item() - ref.item()))
```

The usual PyTorch version follows; from Chapter 1.7 on, programs are written this way:

The network and training loop in idiomatic PyTorch code/u1/mnist_torch.py

```
model = torch.nn.Sequential(
    torch.nn.Linear(784, 256),
    torch.nn.ReLU(),
    torch.nn.Linear(256, 128),
    torch.nn.ReLU(),
    torch.nn.Linear(128, 10),
).to(dev)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
xs, ys = torch.tensor(x, device=dev), torch.tensor(y, device=dev)
rng = np.random.default_rng(1)
start = time.perf_counter()
for _ in range(epochs):
    for idx in batches(len(x), 128, rng):
        idx = torch.from_numpy(idx).to(dev)
        loss = F.cross_entropy(model(xs[idx]), ys[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
device.sync(dev)
seconds = (time.perf_counter() - start) / epochs
with torch.no_grad():
    predictions = model(torch.tensor(x_test, device=dev)).argmax(1).cpu().numpy()
```

Lockstep agreement, then the network on the CPU and on the GPU

```
$ uv run -m u1.mnist_torch
tiny and PyTorch in lockstep for 100 steps (float64): largest loss difference 7.8e-16
PyTorch on cpu: 0.44 s per epoch, test accuracy 97.53%
PyTorch on mps: 2.14 s per epoch, test accuracy 97.57%
```

After a hundred steps the two libraries' losses agree to 10^{-15}. On the CPU, PyTorch and `tiny` take about the same time, since both spend it in the same BLAS library. On the GPU the network trains *more slowly*. Each step launches dozens of small operations, and every launch costs microseconds of overhead in the driver, while the matrices involved, 128 rows by a few hundred columns, are far too small to occupy the GPU's thousands of arithmetic units. The GPU is faster only when each operation has enough work, and Lab 1.5.3 finds where that happens.

> **KEY IDEA**
> A layer preserves the scale of its input when the variance of its weights is the gain of its activation divided by its number of inputs: 2 for ReLU, 1 for `tanh` or no activation. With any other scale, a deep network's activations and gradients vanish or explode before training starts, and the network cannot be trained.

> **IN PYTORCH**
> `torch.nn.Module` collects parameters as `tiny`'s does, and `torch.nn.Linear(n_in, n_out)` holds a weight of shape (n_out, n_in) and computes $xW^T + b$. Its default initialisation is uniform with variance $1/(3n)$, deliberately conservative; `torch.nn.init.kaiming_normal_(w, nonlinearity="relu")` gives He's. `model.train()` and `model.eval()` switch layers such as dropout between their two behaviours, and `model.state_dict()` holds every parameter for saving.

Reading

- Xavier Glorot and Yoshua Bengio, "Understanding the Difficulty of Training Deep Feedforward Neural Networks", AISTATS 2010.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, "Delving Deep into Rectifiers", ICCV 2015, section 2.2.
- Simon Prince, *Understanding Deep Learning*, MIT Press, 2023, chapters 3, 4 and 7. Free online.

Labs

LAB 1.5.1 Predicting the scale — 1.5 hours

Goal. Check the variance argument quantitatively, and extend it to a new activation.

Code. `code/u1/init.py`

1. For each scheme in the program, predict the ratio of successive layers' scales from the variance formula, and compare it with the measurement.
2. Add GELU, $x\Phi(x)$ with Φ the standard normal distribution function. Find by experiment the weight variance that keeps its activations steady, and compare it with $2/n$.

DONE WHEN

- [ ] Your predicted ratios match the measured ones within 5% for the four ReLU schemes.
- [ ] You have GELU's gain, measured to two significant figures.

EVIDENCE FOR THE NOTEBOOK

- The table of predicted and measured ratios, and the GELU experiment.

LAB 1.5.2 Width and depth — 1.5 hours

Goal. Measure how width and depth affect accuracy on MNIST.

Code. `code/u1/mnist_tiny.py`

1. Train MLPs in `tiny` with one hidden layer of 32, 128, 512 and 2048 units, and with two, four and eight hidden layers of 256.
2. Record parameters, test accuracy and seconds per epoch for each.

DONE WHEN

- [ ] Your table covers all seven networks.
- [ ] You can say where extra capacity stopped helping, and whether width or depth was the better use of parameters.

EVIDENCE FOR THE NOTEBOOK

- The table and a plot of accuracy against parameters.

LAB 1.5.3 When the GPU is faster — 1.5 hours

Goal. Find the size at which the GPU overtakes the CPU for this kind of network.

Code. `code/u1/mnist_torch.py`

1. Time one epoch of the PyTorch network on both devices at batch sizes 32, 128, 512 and 2048, and at hidden widths 256, 1024 and 4096.
2. Find the crossover points and relate them to the matrix sizes in Chapter 1.1's matrix-multiplication timings.

DONE WHEN

- [ ] You have both timings for all twelve combinations.
- [ ] You can predict, for a new width and batch size, which device will be faster.

EVIDENCE FOR THE NOTEBOOK

- The timing table and your rule of thumb.

LAB 1.5.4 Inside the network — 1.5 hours

Goal. Look at what the first layer learned and how the hidden units behave.

Code. `code/u1/mnist_tiny.py`

1. Draw 32 of the first layer's 256 weight vectors as 28×28 images, as Chapter 1.3 drew the linear model's templates.
2. Measure the fraction of hidden units that output exactly zero, per layer, on the test set, and find any units that are zero for every input.

DONE WHEN

- [ ] You can describe the kinds of feature the first layer detects, with examples.
- [ ] You have the fraction of zeros in each layer and a count of units that never activate.

EVIDENCE FOR THE NOTEBOOK

- The images, the fractions and the count.

Problem set

1. Show that two stacked linear layers, $W_2^T\left(W_1^T x + b_1\right) + b_2$, equal one linear layer, and give its weight and bias.
2. Count the parameters of an MLP with layer widths 784, 256, 128, 10, and check the program's figure of 235,146.
3. Derive $E\left[\mathrm{ReLU}(z)^2\right] = \frac{1}{2}\,\mathrm{Var}(z)$ for z symmetric about zero. Where does the argument use the symmetry?
4. The output of `tanh` saturates at ± 1. If the pre-activations of a layer have standard deviation 5, what fraction of units have gradients below 0.01?
5. PyTorch's default initialisation has variance $1/(3n)$. By what factor does it shrink a ReLU network's activations per layer, and after 30 layers?

COMPLETION CHECKLIST

- [ ] I can explain why nonlinearities are needed and what ReLU's advantages are.
- [ ] I can derive He and Xavier initialisation from the variance of a layer.
- [ ] I can train an MLP in `tiny` and in PyTorch and show that they agree.
- [ ] I can explain when a GPU is slower than a CPU, and measure where it overtakes.
- [ ] Lab 1.5.1–1.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.6

Generalisation

TIME	5 hours
YOU BUILD	Overfitting produced with 1,000 MNIST images and a network of 670,000 parameters, then reduced with weight decay, dropout, data augmentation and early stopping; and double descent reproduced with random features, the test error peaking where the parameters equal the examples and falling again beyond.
NEEDS	Chapter 1.5.

WHY IT MATTERS

A model must predict well on data it has not seen, since doing well on the training data alone is no more than a lookup table achieves. A network large enough to be useful is also large enough to memorise its training set, noise and all, and most real projects succeed or fail on the gap between training and test performance. This chapter produces that gap on purpose, closes it with the standard tools, and then shows an effect that classical statistics did not predict, in which models past a certain size can generalise better as they grow.

SKIP TEST

1. What are the training, validation and test sets each for, and why must the test set not be used to choose settings?
2. What does dropout do during training and at test time, and why are the kept activations divided by $1 - p$?
3. Why does training on randomly shifted copies of images reduce overfitting?
4. What is double descent, and at what model size does the test error peak?

If all four are easy, read the section on double descent and do Labs 1.6.3 and 1.6.4.

Core ideas

The generalisation gap

The data a model will meet in use are assumed to come from some distribution, and the training set is a sample from it. The quantity of interest is the expected loss on fresh samples, which cannot be computed and can only be estimated on held-out data. The difference between that and the training loss is the *generalisation gap*.

An honest estimate depends on how the data are split and used. The *training set* fits the parameters. The *validation set* chooses everything else: architecture, learning rate, regularisation strength, the epoch to stop at. Each such choice fits the validation set a little, so its score becomes optimistic too. The *test set* is used once, at the end, for the number you report. Any path by which test information reaches a decision, including looking at test scores while tuning, is *leakage*, and makes the reported number unreliable.

Classical statistics frames the gap as a trade-off. A model with too little capacity cannot fit the true pattern and errs systematically, which is *bias*. A model with too much fits the particular sample, noise included, so its predictions vary from sample to sample, which is *variance*. Test error is then U-shaped in model size, lowest somewhere in between.

Overfitting a small training set

Training a wider version of Chapter 1.5's network on a thousand images makes overfitting easy to see. With two hidden layers of 512 units it has 670,000 parameters, nearly 700 for every training image.

Training one variant on 1,000 images, recording training loss and validation accuracy each epoch — code/u1/overfit.py

```
def train(
    x,
    y,
    x_val,
    y_val,
    epochs: int,
    weight_decay: float = 0.0,
    dropout: float = 0.0,
    augment: bool = False,
    seed: int = 1,
):
    rng = np.random.default_rng(seed)
    model = nn.mlp([784, 512, 512, 10], rng, dropout=dropout)
    opt = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=weight_decay)
    history = []
    for epoch in range(epochs):
        losses = []
        for idx in batches(len(x), 100, rng):
            xb = shift(x[idx], rng) if augment else x[idx]
            loss = cross_entropy(model(Tensor(xb)), y[idx])
            opt.zero_grad()
            loss.backward()
            opt.step()
            losses.append(loss.item())
        history.append((float(np.mean(losses)), accuracy(model, x_val, y_val)))
    return model, history
```

Five variants, 150 epochs each

```
$ uv run -m u1.overfit
1,000 training images, 150 epochs; validation on 5,000, test on 10,000
                          train loss  final val  best val  at epoch     test
no regularisation             0.0000     90.44%    90.48%       146   91.07%
weight decay 0.5              0.0001     90.58%    90.88%        84   91.43%
dropout 0.5                   0.0214     90.92%    91.92%       101   91.39%
shifts of up to 2 pixels      0.0393     92.66%    94.54%       137   93.27%
all three                     0.1715     94.90%    95.44%       148   95.39%
```

Without regularisation the training loss reaches zero to four decimal places, so the network has memorised its thousand images. Its validation accuracy settles near 90.5%, far below the 97.9% that Chapter 1.5's network reaches with all 55,000 training images. Each regulariser below narrows the gap a little, and together they reach 95% from the same thousand images (Figure 1.6.1). The test column is printed so that the book can show it. In your own work, compare the variants on validation alone and test only the chosen one.

Four regularisers

Weight decay pulls every weight towards zero, preferring simpler functions with smaller weights. From the probabilistic view of Chapter 1.3 it is a prior: a Gaussian prior on the weights adds $\lambda\|\theta\|^2$ to the negative log-likelihood, and the minimiser is the *maximum a posteriori* estimate. With AdamW the shrinkage per step is the product of the learning rate and the decay, here 5×10^{-4}. Over 1,500 steps that is modest, and so is its effect on this problem.

Dropout sets each hidden activation to zero with probability p during training, independently at every step, so that no unit can rely on any particular other unit being present. Dividing the surviving activations by $1 - p$ keeps their expected value unchanged, so at test time, when nothing is dropped, the network needs no adjustment:

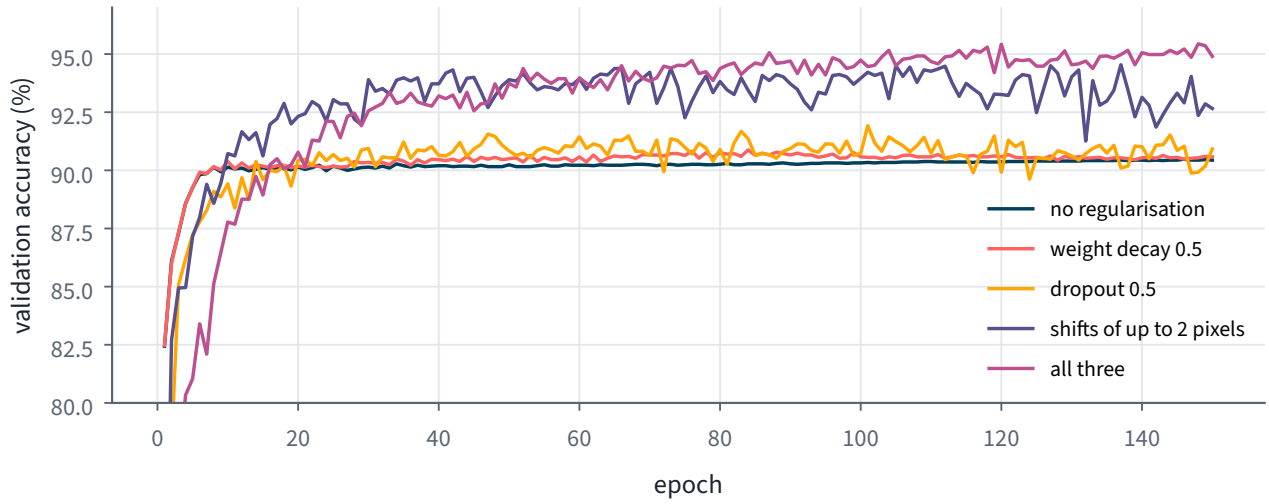


Figure 1.6.1. Validation accuracy per epoch for the five variants. Augmentation helps most; the combination keeps improving long after the unregularised network has stopped.

Inverted dropout, active only in training mode — code/aif/tiny/nn.py

```
class Dropout(Module):
    def __init__(self, p: float, rng: np.random.Generator):
        self.p, self.rng = p, rng

    def forward(self, x: Tensor) → Tensor:
        if not self.training or self.p == 0.0:
            return x
        keep = (self.rng.random(x.shape) ⩾ self.p).astype(x.data.dtype)
        return x * (keep / (1.0 - self.p))
```

Data augmentation trains on transformed copies of the data that should not change the answer, here shifts of up to two pixels in each direction. It teaches the network an invariance the data imply but a thousand images do not show, and is the most effective of the four here, as it usually is wherever such invariances are known:

Random shifts, applied afresh to every batch — code/u1/overfit.py

```
def shift(images: np.ndarray, rng: np.random.Generator, pixels: int = 2) → np.ndarray:
    grid = images.reshape(-1, 28, 28)
    out = np.empty_like(grid)
    for i, img in enumerate(grid):
        dy, dx = rng.integers(-pixels, pixels + 1, 2)
        out[i] = np.roll(np.roll(img, dy, axis=0), dx, axis=1)
    return out.reshape(len(images), -1)
```

Early stopping keeps the parameters from the epoch with the best validation score. The console's "best val" column shows what it would gain over the final epoch for each variant: a little for the unregularised network, more for those whose validation accuracy fluctuates.

PITFALL

Dropout and batch normalisation behave differently in training and in evaluation. Forgetting `model.eval()` before measuring leaves dropout on, and the measured accuracy is lower and noisy; forgetting `model.train()` afterwards trains without it. Every evaluation function in this course switches modes and restores them.

The precision of a test score

A test accuracy is an estimate from a finite sample. If a model's true accuracy is p and the test set has n independent examples, the measured accuracy has a standard error of $\sqrt{p(1-p)/n}$. For $p = 0.95$ and $n = 10,000$ that is 0.22 percentage points, so a 95% confidence interval spans about ± 0.43 points. Differences

between models smaller than that, such as weight decay's 91.43% against dropout's 91.39% above, are not evidence that one model is better. Unit 3 returns to the statistics of evaluation. Until then, ask how large a difference must be before it means something.

Double descent

The U-shaped picture predicts that a model with far more parameters than training examples should generalise terribly. Modern networks, with millions of parameters trained on thousands of examples, routinely do not. A simple setting shows how this can happen. Fix p random features of each image, $\varphi(x) = \mathrm{ReLU}(Wx)/\sqrt{p}$ with W random and never trained, and fit a linear model on them by least squares. With p smaller than the number of examples n the fit is the usual least-squares solution. With $p > n$ infinitely many solutions fit the training data exactly, and the pseudo-inverse picks the one with the smallest norm.

Random ReLU features and the minimum-norm least-squares fit — code/u1/double_descent.py

```
def random_features(x: np.ndarray, w: np.ndarray) → np.ndarray:
    return np.maximum(x @ w, 0) / np.sqrt(w.shape[1])


def fit_min_norm(phi: np.ndarray, targets: np.ndarray) → np.ndarray:
    return np.linalg.pinv(phi, rcond=1e-10) @ targets
```

Four hundred training images get one-hot targets, a fifth of them with random labels to make the effect clear:

Randomising a fifth of the labels — code/u1/double_descent.py

```
noisy = y.copy()
flip = rng.random(n) < 0.2
noisy[flip] = rng.integers(0, 10, flip.sum())
targets = np.eye(10)[noisy]
```

Test error and weight norm against the number of features

```
$ uv run -m u1.double_descent
400 training images, 20% of labels randomised; min-norm least squares on random ReLU
    features
 features  train error  test error  test MSE  weight norm
       20        51.0%       55.2%      0.77          0.2
       50        33.8%       46.5%      0.72          0.4
      100        17.9%       41.4%      0.96          3.0
      200         3.9%       47.4%      1.09          2.5
      300         0.5%       59.9%      2.24          4.6
      350         0.1%       71.9%      5.08          7.2
      380         0.0%       77.0%     11.81         11.5
      400         0.0%       87.9%   1180.33        118.0
      420         0.0%       78.5%     15.37         13.0
      450         0.0%       74.1%      5.71          7.8
      500         0.0%       66.1%      3.62          6.1
      600         0.0%       55.8%      1.89          4.4
      800         0.0%       48.5%      1.25          3.6
     1200         0.0%       39.5%      0.88          3.1
     2000         0.0%       34.7%      0.72          2.8
     4000         0.0%       30.9%      0.63          2.6
     8000         0.0%       29.3%      0.59          2.5
```

The test error first follows the classical U, lowest near 100 features, then climbs to a peak of 88% at exactly 400 features, where the number of parameters equals the number of examples. There the model can just interpolate the training data, including its noisy labels. It has only one way to do so, which needs enormous weights, and the weight norm peaks at 118. Beyond the threshold, many solutions interpolate the data, the minimum-norm one among them is smooth, and the test error falls again, to 29% at 8,000 features, well below the classical minimum. This is *double descent*. Large networks trained by gradient descent behave similarly, with the optimiser's own preference for small, simple solutions playing the part of the minimum norm.

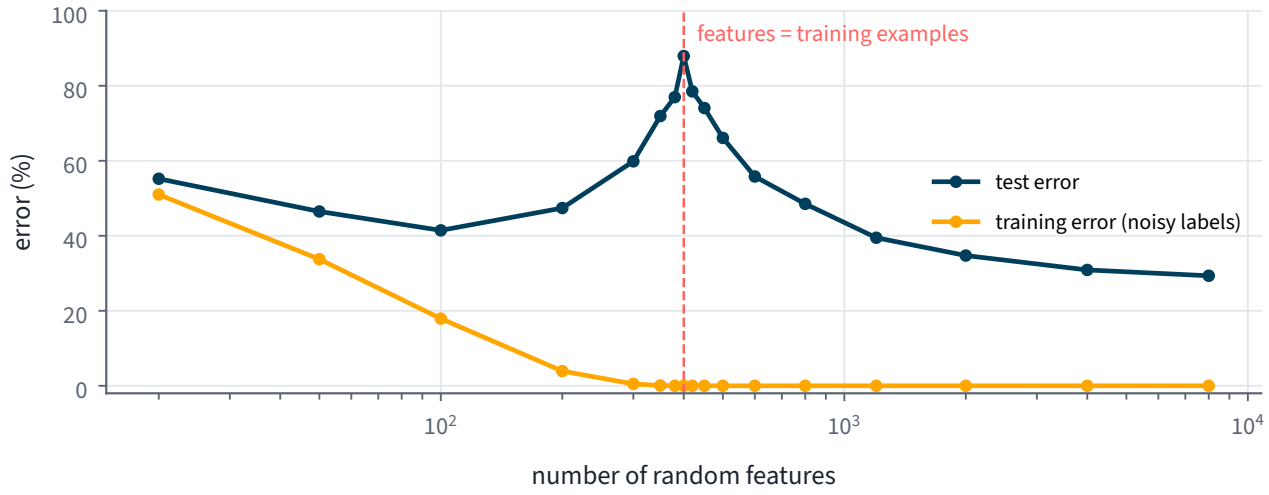


Figure 1.6.2. Test error against the number of random features, on a logarithmic scale. The error falls, rises to a sharp peak where features equal examples, and falls again, below the earlier minimum.

KEY IDEA

Generalisation depends on the data, the model's capacity and the inductive bias of the training procedure together. Regularisers and augmentation shrink the gap by encoding prior knowledge about good solutions. Past the interpolation threshold, size itself can help, because among the many functions that fit, training finds simple ones.

IN PYTORCH

`torch.nn.Dropout(p)` implements inverted dropout and obeys `model.eval()`. Weight decay is an optimiser argument, `torch.optim.AdamW(params, weight_decay=0.1)`. Image augmentations live in `torchvision.transforms`, and `torch.linalg.pinv` computes the pseudo-inverse used above.

Reading

- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever and Ruslan Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *JMLR* 15, 2014.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht and Oriol Vinyals, "Understanding Deep Learning Requires Rethinking Generalization", ICLR 2017.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma and Soumik Mandal, "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-off", *PNAS* 116(32), 2019.
- Preetum Nakkiran and others, "Deep Double Descent: Where Bigger Models and More Data Hurt", ICLR 2020.

Labs

LAB 1.6.1 Learning curves — 1 hour

Goal. Measure how accuracy grows with data, the most reliable way to improve a model.

Code. `code/u1/overfit.py`

1. Train the unregularised network on 500, 1,000, 5,000, 20,000 and 55,000 images, with as many epochs as it takes for the training loss to stop falling.
2. Plot validation error against training-set size on logarithmic axes and fit a straight line.

DONE WHEN

- [ ] You have five points and a fitted slope.
- [ ] You can predict from the fit how many images would halve the error of the 1,000-image model, and say how much you trust the prediction.

EVIDENCE FOR THE NOTEBOOK

- The plot, the fit and the prediction.

LAB 1.6.2 Tuning without leaking

1.5 hours

Goal. Choose regularisation strengths by validation and report a single test score.

Code. `code/u1/overfit.py`

1. On the 1,000-image problem, search weight decay in {0, 0.1, 0.5, 2} and dropout in {0, 0.25, 0.5}, with augmentation on, judging by the best validation accuracy.
2. Test the chosen setting once. Give a 95% interval for its test accuracy.

DONE WHEN

- [ ] Your search covers all twelve settings, judged only on validation.
- [ ] Your report has one test score with its interval.

EVIDENCE FOR THE NOTEBOOK

- The validation table, the choice and the test result.

LAB 1.6.3 Memorising noise

1 hour

Goal. Reproduce the observation that started the rethinking of generalisation.

Code. `code/u1/overfit.py`

1. Train the unregularised network on 1,000 images whose labels are replaced by random ones.
2. Record the epochs needed to reach 100% training accuracy, compared with real labels, and the validation accuracy.

DONE WHEN

- [ ] The network fits random labels perfectly, and you have the number of epochs for both cases.
- [ ] You can explain what this implies about parameter counts as a measure of overfitting.

EVIDENCE FOR THE NOTEBOOK

- Both learning curves and the explanation.

LAB 1.6.4 What moves the peak

1.5 hours

Goal. Explore double descent's dependence on noise and regularisation.

Code. `code/u1/double_descent.py`

1. Rerun the sweep with label noise of 0%, 20% and 40%.
2. Replace the pseudo-inverse with ridge regression, $\left(\Phi^T\Phi + \lambda I\right)^{-1}\Phi^T Y$, at three values of λ, and rerun.

DONE WHEN

- [ ] You have six curves of test error against features.
- [ ] You can say how noise and λ change the peak, and why ridge regression suppresses it.

EVIDENCE FOR THE NOTEBOOK

- The curves and your explanation.

Problem set

1. A model scores 96.0% and another 96.3% on a test set of 10,000 images. Is the second better? What if the same test images were used, and the two disagree on only 150 of them?
2. Show that minimising the negative log-likelihood plus $\lambda\|\theta\|^2$ finds the most probable parameters under a Gaussian prior. What is the prior's variance?
3. With dropout $p = 0.5$ on a layer of 512 units, how many different sub-networks can a training step use? Why might averaging over them help?
4. Name an augmentation that would be harmful for MNIST, and one that would be harmless for photographs of animals but harmful for photographs of text.
5. At exactly $p = n$ features, why does the minimum-norm solution have a very large norm? Relate your answer to the smallest singular value of the feature matrix.

COMPLETION CHECKLIST

- [ ] I can explain the roles of the training, validation and test sets, and recognise leakage.
- [ ] I can apply weight decay, dropout, augmentation and early stopping, and explain how each works.
- [ ] I can give the uncertainty of a test score and judge whether a difference is meaningful.
- [ ] I can explain double descent and the role of the interpolation threshold.
- [ ] Lab 1.6.1–1.6.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.7

Training deep networks

TIME	6 hours
YOU BUILD	A 30-layer network that will not train, measured layer by layer, and made to train by initialisation, normalisation and residual connections; batch, layer and RMS normalisation written by hand and matched to PyTorch's; and three planted bugs diagnosed from training statistics alone.
NEEDS	Chapters 1.5 and 1.6. From here on, programs use PyTorch.

WHY IT MATTERS

Depth gives neural networks much of their capability and also makes them hard to train, because signals and gradients pass through every layer and small distortions compound. Two inventions made very deep networks routine, normalisation layers and residual connections, and both are central to the transformer that Unit 2 builds. This chapter shows the failure they fix and builds them. It then turns to the other half of training deep networks, finding out what is wrong when a run misbehaves.

SKIP TEST

1. A 30-layer ReLU network uses PyTorch's default initialisation. What happens to the gradient of its first layer, and why?
2. How does batch normalisation behave differently in training and evaluation, and what goes wrong with a batch of one?
3. Why is a pre-norm residual network easier to train than a post-norm one?
4. Training loss rises steadily from the first step and the gradient norm grows without bound. Name two bugs that would do this.

If all four are easy, read the sections on residual connections and debugging, and do Labs 1.7.2 and 1.7.3.

Core ideas

Gradients through many layers

By the chain rule, the gradient reaching layer l is the gradient at the output multiplied by the Jacobian of every layer after it. If each multiplication shrinks the gradient by a factor, thirty of them shrink it by that factor to the thirtieth power; if each grows it, it explodes. Chapter 1.5 showed the forward version of this arithmetic. The program measures the backward version in four 30-layer networks of width 256 on MNIST, each trained for two epochs:

Four ways to stack thirty layers code/u1/deep.py

```python
class DeepNet(torch.nn.Module):
    def __init__(self, depth: int, width: int, kind: str):
        super().__init__()
        self.kind = kind
        self.stem = torch.nn.Linear(784, width)
        self.layers = torch.nn.ModuleList(torch.nn.Linear(width, width) for _ in range(depth))
        self.norms = torch.nn.ModuleList(torch.nn.LayerNorm(width) for _ in range(depth))
        self.head = torch.nn.Linear(width, 10)
        if kind != "plain, default init":
            for layer in self.layers:
                torch.nn.init.kaiming_normal_(layer.weight, nonlinearity="relu")
                torch.nn.init.zeros_(layer.bias)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.stem(x)
        for layer, norm in zip(self.layers, self.norms):
            if self.kind.startswith("plain"):
                h = F.relu(layer(h))
            elif self.kind == "post-norm":
                h = norm(h + layer(F.relu(h)))
            else:
                h = h + layer(F.relu(norm(h))) / math.sqrt(len(self.layers))
        return self.head(h)
```

The gradient norm of each layer's weights for one batch code/u1/deep.py

```python
def gradient_norms(model: DeepNet, x: torch.Tensor, y: torch.Tensor) -> list[float]:
    model.zero_grad()
    F.cross_entropy(model(x), y).backward()
    return [layer.weight.grad.norm().item() for layer in model.layers]
```

Thirty layers four ways

```
$ uv run -m u1.deep
30 hidden layers of 256 units on MNIST, AdamW at lr 0.001, 2 epochs, seed 0
network                grad norm, layer 1  layer 30  loss at 100  val accuracy
plain, default init              2.22e-12  7.33e-03        2.302        11.06%
plain, He init                   1.25e-01  1.13e-01        0.974        40.74%
post-norm                        2.93e-01  6.29e-01        1.026        63.40%
pre-norm residual                3.12e-01  1.66e-01        0.254        96.60%
```

With PyTorch's default initialisation, whose variance is a sixth of He's, the first layer's gradient is 10^{-12}, so almost nothing reaches it. The loss stays at $\ln 10$ and the accuracy stays at chance. He initialisation keeps the gradients level at the start, and the network learns, but slowly and unreliably. With other seeds (`--seed`), its

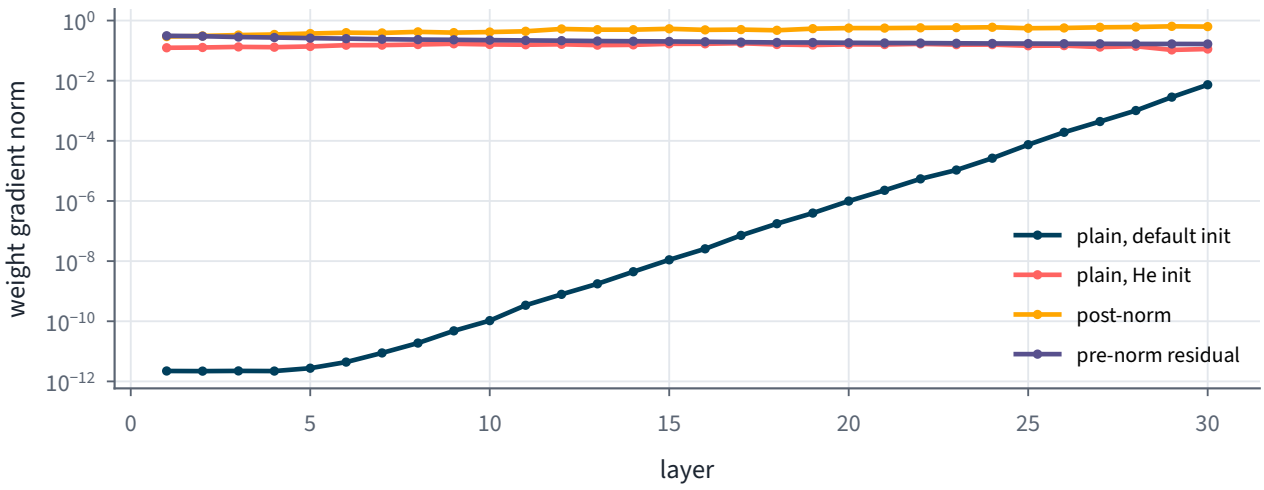


Figure 1.7.1. Gradient norm of each layer's weights at initialisation, on a logarithmic scale. With PyTorch's default initialisation the first layer receives a gradient nearly ten orders of magnitude smaller than the last.

accuracy after two epochs ranges from about 40% to about 85%, because nothing stops the scales drifting as the weights change. Initialisation sets only the starting point.

Normalisation layers

A *normalisation layer* rescales activations to zero mean and unit variance at every step of training, wherever the weights have drifted, and then applies a learned scale γ and shift β so that the network can undo it where that helps:

$$y = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta.$$

The variants differ in which numbers the statistics μ and σ^2 are computed over. *Batch normalisation* computes them per feature, over the examples in the batch. That makes each example's output depend on the others in its batch, and at evaluation time, when there may be no batch, it uses running averages accumulated during training instead:

Batch normalisation with running statistics for evaluation — code/u1/norms.py

```
class BatchNorm(torch.nn.Module):
    def __init__(self, features: int, momentum: float = 0.1, eps: float = 1e-5):
        super().__init__()
        self.weight = torch.nn.Parameter(torch.ones(features))
        self.bias = torch.nn.Parameter(torch.zeros(features))
        self.register_buffer("running_mean", torch.zeros(features))
        self.register_buffer("running_var", torch.ones(features))
        self.momentum, self.eps = momentum, eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if self.training:
            mean, var = x.mean(0), x.var(0, unbiased=False)
            with torch.no_grad():
                n = x.shape[0]
                self.running_mean.lerp_(mean, self.momentum)
                self.running_var.lerp_(var * n / (n - 1), self.momentum)
        else:
            mean, var = self.running_mean, self.running_var
        return (x - mean) / torch.sqrt(var + self.eps) * self.weight + self.bias
```

Layer normalisation computes them per example, over its features, so each example is treated alone and training and evaluation behave identically. It is the usual choice for sequences, where batches mix lengths and positions. *RMS normalisation* drops the mean and the shift and divides by the root-mean-square alone, which is cheaper and works as well in transformers, including the ones in this course:

Layer normalisation and RMS normalisation — code/u1/norms.py

```
class LayerNorm(torch.nn.Module):
    def __init__(self, features: int, eps: float = 1e-5):
        super().__init__()
        self.weight = torch.nn.Parameter(torch.ones(features))
        self.bias = torch.nn.Parameter(torch.zeros(features))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        mean = x.mean(-1, keepdim=True)
        var = x.var(-1, unbiased=False, keepdim=True)
        return (x - mean) / torch.sqrt(var + self.eps) * self.weight + self.bias


class RMSNorm(torch.nn.Module):
    def __init__(self, features: int, eps: float = 1e-5):
        super().__init__()
        self.weight = torch.nn.Parameter(torch.ones(features))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) * self.weight
```

All three match `torch.nn`, outputs, gradients and running statistics

```
$ uv run -m u1.norms
BatchNorm  matches torch.nn: largest difference 9.1e-14
LayerNorm  matches torch.nn: largest difference 5.7e-14
RMSNorm    matches torch.nn: largest difference 0.0e+00
```

The batch statistics use the biased variance, dividing by n, while the running variance is updated with the unbiased one, dividing by $n - 1$. This is PyTorch's convention, and matching an implementation to the last digit is what catches details like it.

Residual connections

A *residual connection* adds a block's input to its output: $h_{l+1} = h_l + f_l(h_l)$. Each block then learns a correction to an identity map instead of a whole new representation, and the derivative of h_{l+1} with respect to h_l is $I + \partial f_l / \partial h_l$. The identity term carries gradients straight from the loss to every layer, however deep the network. Dividing each branch by $\sqrt{L}$ keeps the sum of thirty branches from growing with depth.

Where the normalisation goes matters. *Post-norm* adds the branch to the residual path and then normalises the sum, $h_{l+1} = \mathrm{Norm}(h_l + f_l(h_l))$, as the original transformer did. *Pre-norm* normalises the branch's input and leaves the residual path clean, $h_{l+1} = h_l + f_l(\mathrm{Norm}(h_l))$. At a learning rate of 10^{-3} the post-norm network is unstable. It reaches 63% in the console, and with seeds 1, 2 and 3 it reaches 57% once and stays at chance twice, while the pre-norm residual network reaches 96.6% in two epochs. Post-norm can be trained with care: at a learning rate of 10^{-4} it reaches 94% in one epoch, and with a warm-up it tolerates higher rates. Pre-norm, which every transformer in this course uses, needs no such care.

> **KEY IDEA**
> Normalisation keeps every layer's activations at a healthy scale throughout training, and residual connections give gradients a direct path through any depth. Together, in the pre-norm arrangement, they make a deep network nearly as easy to train as a shallow one.

When training goes wrong

Most failed training runs do not crash. They train to a disappointing number, or plateau, or diverge after hours, and the cause can be in the data, the model, the loss, the optimiser, or the loop that connects them. They can be debugged methodically.

Before training, check that the initial loss is what chance predicts, $\ln K$ for K classes, and that the model can drive the loss to nearly zero on a single small batch. A model that cannot overfit one batch has a bug, because a single batch is well within its capacity. During training, log more than the loss: the gradient norm, the scale of activations and logits, the learning rate, and an accuracy. Each common bug leaves a

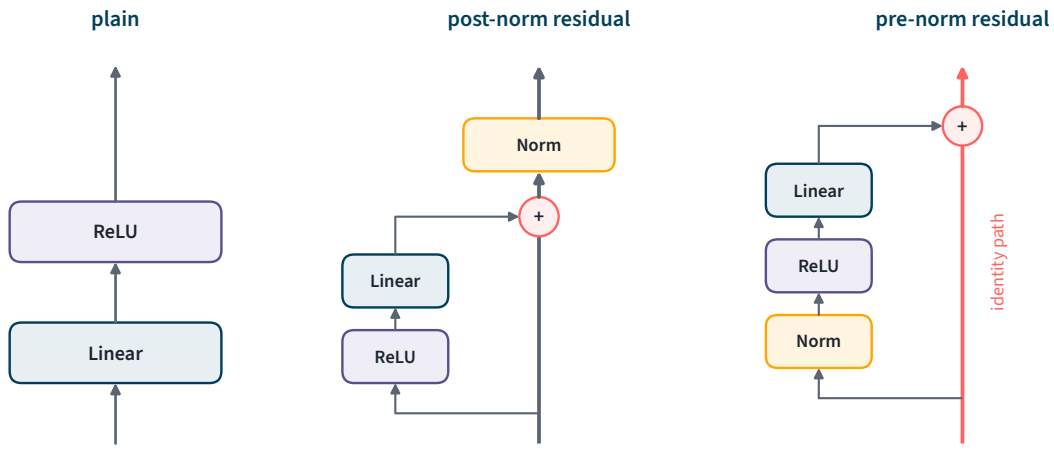


Figure 1.7.2. Three ways to build a layer: plain, post-norm residual and pre-norm residual. Only the last leaves an unobstructed path from output to input; post-norm puts a normalisation on it after every layer.

characteristic pattern in these numbers. Change one thing at a time, keep a known-good run to compare with, and distrust any result that you cannot reproduce.

`u1/mystery.py` contains three runs of Chapter 1.5's network, labelled A, B and C, each with one bug planted in it, and a clean run for comparison. Lab 1.7.3 asks you to diagnose them from their statistics before reading the code. This is the clean run:

The clean run, the reference for reading the mystery runs

```
$ uv run -m u1.mystery
 step     loss  batch acc  grad norm  |logits| max
    0    2.317       7.0%      0.676         0.367
   25    0.778      76.6%       3.79          31.1
   50    0.255      92.2%       0.88          16.5
  100    0.175      94.5%      0.742            16
  200    0.141      96.1%      0.763          14.9
  400    0.155      92.2%      0.998            19
  600    0.095      94.5%      0.847          21.9
validation accuracy 96.08%
```

Its loss starts at 2.317, close to $\ln 10$. The gradient norm settles below 1 after an early peak, the logits stay in the tens, and the validation accuracy reaches 96%.

PITFALL

A bug that only lowers final accuracy by a few points is the hardest kind to find, because the run looks healthy. Compare against a baseline you trust, from a paper, a library or your own earlier run, and treat any unexplained shortfall as a bug until shown otherwise.

IN PYTORCH

`torch.nn.BatchNorm1d`, `LayerNorm` and `RMSNorm` are this chapter's layers. `module.register_forward_hook` and `register_full_backward_hook` attach functions that see every layer's activations and gradients, the basis of any training monitor. `torch.autograd.set_detect_anomaly(True)` reports the forward operation whose backward pass first produced a NaN, at a large cost in speed.

Reading

- Sergey Ioffe and Christian Szegedy, "Batch Normalization", ICML 2015.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, "Deep Residual Learning for Image Recognition", CVPR 2016.
- Ruibin Xiong and others, "On Layer Normalization in the Transformer Architecture", ICML 2020: pre-norm against post-norm.
- Andrej Karpathy, "A Recipe for Training Neural Networks", 2019, a practitioner's checklist.

Labs

LAB 1.7.1 Measuring usable depth — 1.5 hours

Goal. Find how deep each kind of network can usefully go.

Code. `code/u1/deep.py`

1. Train the plain He-initialised network and the pre-norm residual network at depths 5, 10, 20, 30 and 50, two epochs each, with three seeds per setting: `--kinds "plain, He init" "pre-norm residual" --depth 5 --seed 1`, and so on.
2. Plot validation accuracy against depth, with the spread across seeds.

DONE WHEN

- [ ] You have both curves with their spreads.
- [ ] You can state the depth beyond which the plain network becomes unreliable on your machine.

EVIDENCE FOR THE NOTEBOOK

- The plot and the table behind it.

LAB 1.7.2 Training post-norm — 1 hour

Goal. Make the post-norm network train, and measure what it needed.

Code. `code/u1/deep.py`

1. Train the post-norm network at learning rates from 3×10^{-5} to 10^{-3}, with and without a linear warm-up over 500 steps: `--kinds post-norm --lr 3e-4 --warmup 500`, and so on.
2. Record the validation accuracy of each.

DONE WHEN

- [ ] Your table shows the highest learning rate at which post-norm trains, with and without warm-up.
- [ ] You can explain the difference using the gradient norms at the start of training.

EVIDENCE FOR THE NOTEBOOK

- The table and the gradient-norm comparison.

LAB 1.7.3 Three mystery runs — 1.5 hours

Goal. Diagnose training bugs from their statistics alone.

Code. `code/u1/mystery.py`

1. Run `uv run -m u1.mystery --case A`, then B, then C. Their output is below.
2. For each, write down which numbers depart from the clean run's, what bug would cause that pattern, and how you would confirm it. Only then read `run` and check.

Case A

```
$ uv run -m u1.mystery --case A
 step     loss  batch acc  grad norm  |logits| max
    0    2.317       7.0%      0.676         0.367
   25    2.867      10.2%       90.6          54.4
   50    4.184       9.4%        146          5.37
  100   28.231       9.4%        147          54.1
  200  127.253      14.1%        148           143
  400  585.150       8.6%        149           840
  600  730.289      11.7%        154      1.59e+03
validation accuracy 9.62%
```

Case B

```
$ uv run -m u1.mystery --case B
 step     loss  batch acc  grad norm  |logits| max
    0    2.317      10.9%      0.473         0.319
   25    2.308       8.6%      0.468         0.374
   50    2.304      10.9%      0.288         0.272
  100    2.298      10.2%      0.207         0.265
  200    2.310       8.6%      0.147         0.219
  400    2.301      14.1%      0.119         0.192
  600    2.307       7.8%      0.185         0.252
validation accuracy 11.56%
```

Case C

```
$ uv run -m u1.mystery --case C
 step     loss  batch acc  grad norm  |logits| max
    0    2.304       7.0%     0.0668         0.367
   25    2.261      53.1%      0.164          1.64
   50    1.870      66.4%      0.326          11.9
  100    1.656      81.2%      0.224            29
  200    1.691      77.3%      0.153            26
  400    1.667      79.7%      0.172          31.7
  600    1.646      82.0%      0.181          38.9
validation accuracy 76.06%
```

DONE WHEN

- [ ] You named all three bugs before reading the code.
- [ ] For each, you can point to the statistic that gave it away.

EVIDENCE FOR THE NOTEBOOK

- Your three written diagnoses, dated before you read the code.

LAB 1.7.4 A training monitor — 2 hours

Goal. Build a monitor of per-layer statistics for use on later runs.

Code. `code/u1/deep.py`

1. Write a class that attaches forward and backward hooks to every `Linear` layer of a model and records, each n steps, the standard deviation of its outputs and the norm of its weight gradient.
2. Use it on the four deep networks and draw the statistics as heat maps of layer against step.

DONE WHEN

- [ ] Your monitor works on any `torch.nn.Module` without changes to the model.
- [ ] Your heat maps show the difference between the plain and pre-norm networks over training as well as at initialisation.

EVIDENCE FOR THE NOTEBOOK

- The monitor's code and the heat maps.

Problem set

1. For a residual network $h_{l+1} = h_l + f_l(h_l)$, write $\partial h_L / \partial h_0$ as a product, and expand it to show the term that passes the gradient unchanged.
2. Batch normalisation with a batch of one example: what does it output during training, and why is the network then unable to learn?

3. Why does RMS normalisation work almost as well as layer normalisation in transformers? What does dropping the mean give up?
4. A residual network adds 30 branch outputs of variance v each to an input of variance 1. Without the $1/\sqrt{L}$ scaling, what is the output variance? With it?
5. A run's loss is fine for 10,000 steps, then becomes NaN. List four possible causes and the first thing you would check for each.

COMPLETION CHECKLIST

- ☐ I can explain vanishing and exploding gradients, and measure them layer by layer.
- ☐ I can implement batch, layer and RMS normalisation and explain how they differ.
- ☐ I can explain residual connections and why pre-norm trains more easily than post-norm.
- ☐ I can debug a training run systematically from its statistics.
- ☐ Lab 1.7.1–1.7.4 done-when criteria all hold, with evidence filed.

CHAPTER 1.8

Representations

TIME	7 hours
YOU BUILD	A convolutional residual network trained on CIFAR-10 to over 90% in about twenty minutes; the space of features it learned, examined through nearest neighbours; contrastive learning, which learns features from unlabelled images by telling two views of the same image apart from all others; and a linear probe that measures how well each set of features separates the classes.
NEEDS	Chapters 1.6 and 1.7.

WHY IT MATTERS

A trained network is a classifier on top of a *representation*. Everything below the last layer turns raw input into features in which the classes are easy to separate, and those features are often more useful than the classifier. They let similar inputs be found by comparing vectors, they transfer to tasks the network was never trained on, and, as this chapter shows, they can be learned without any labels at all. Unit 2′s token embeddings and Chapter 3.7′s text embeddings for retrieval are representations of the same kind, and Chapter 3.7 trains its embeddings with this chapter's loss.

SKIP TEST

1. How many weights does a 3×3 convolution from 64 to 128 channels have, and why so few compared with a dense layer between the same feature maps?
2. What does data augmentation do to the training set, in terms of Chapter 1.6′s ideas?
3. Write down a contrastive loss for two views of each of N images, and say what it rewards.
4. What is a linear probe, and why is it used to compare representations?

If all four are easy, read the section on contrastive learning and do Labs 1.8.3 and 1.8.4.

Core ideas

Convolutions

Chapter 1.5′s networks treated an image as a list of 784 numbers, and would learn exactly as well if the pixels were shuffled in the same way for every image. That discards two facts about images: nearby pixels are related, and the same pattern means the same thing wherever it appears. A *convolutional layer* builds both facts in. Each output is computed from a small neighbourhood of the input, here 3×3 pixels across all input channels, and the same weights, the *kernel*, are applied at every position:

$$y_{c,i,j} = b_c + \sum_{c'} \sum_{u,v \in \{-1,0,1\}} w_{c,c',u,v} x_{c',i+u,j+v}.$$

A layer from 64 to 128 channels has $64 \times 128 \times 9 = 73\{,\}728$ weights whatever the image's size. A dense layer between the same 32×32 feature maps would need over 8 billion. Sharing weights across positions makes the layer *equivariant* to translation: shift the input and the output shifts with it. Stacking layers widens what each unit sees, its *receptive field*, and *pooling*, which keeps the largest value in each 2×2 window, halves the resolution so that later layers see more of the image for the same cost.

A network for CIFAR-10

CIFAR-10 has 60,000 colour images of 32×32 pixels in ten classes: aeroplanes, cars, birds, cats, deer, dogs, frogs, horses, ships and lorries. This chapter's network is a small residual network known as ResNet9, from

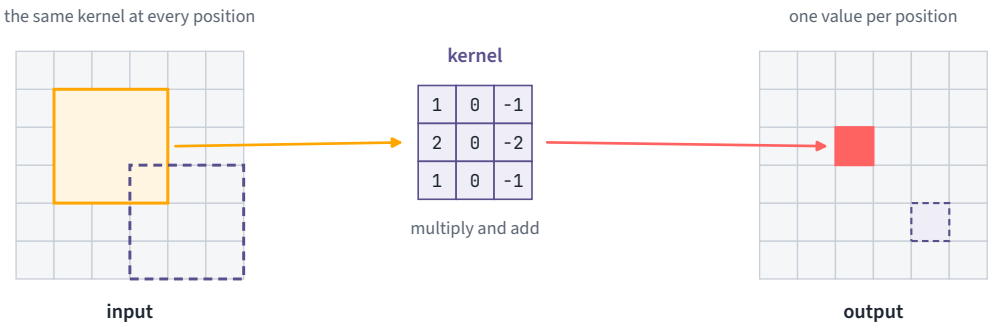


Figure 1.8.1. A 3×3 convolution: each output value is the weighted sum of a neighbourhood of the input, with the same kernel at every position.

David Page's work on training CIFAR-10 quickly. Its building block is a convolution followed by Chapter 1.7's batch normalisation and a ReLU, and two such blocks with a residual connection around them:

Convolution, normalisation, activation; and a residual pair — code/u1/vision.py

```
def conv_bn(c_in: int, c_out: int) -> nn.Sequential:
    return nn.Sequential(
        nn.Conv2d(c_in, c_out, 3, padding=1, bias=False), nn.BatchNorm2d(c_out), nn.ReLU()
    )


class Residual(nn.Module):
    def __init__(self, channels: int):
        super().__init__()
        self.body = nn.Sequential(conv_bn(channels, channels), conv_bn(channels, channels))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.body(x)
```

ResNet9: four stages of growing width, and a linear classifier — code/u1/vision.py

```
class ResNet9(nn.Module):
    def __init__(self, widths=(64, 128, 256, 512), classes: int = 10):
        super().__init__()
        w1, w2, w3, w4 = widths
        self.features = nn.Sequential(
            conv_bn(3, w1),
            conv_bn(w1, w2),
            nn.MaxPool2d(2),
            Residual(w2),
            conv_bn(w2, w3),
            nn.MaxPool2d(2),
            conv_bn(w3, w4),
            nn.MaxPool2d(2),
            Residual(w4),
            nn.AdaptiveMaxPool2d(1),
            nn.Flatten(),
        )
        self.head = nn.Linear(w4, classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x)) * 0.125
```

The network ends by taking each channel's largest value over the whole image, which makes its 512 features independent of where in the image a pattern appeared, and a linear layer turns them into ten logits. Those logits are scaled by $1/8$, because the max-pooled features are large, and without the scale the first steps at a learning rate of 0.2 produce huge logits and diverge.

Fifty thousand images are few for 6.6 million parameters. *Data augmentation* generates more. Every time an image is used, it is padded, cropped at a random offset and flipped left to right at random. A cat shifted

by three pixels or seen in a mirror is still a cat, so the network is trained on many plausible images for each real one. This is regularisation in the sense of Chapter 1.6, and it encodes knowledge about the task.

Random crops and flips for a whole batch at once, on the GPU — code/u1/vision.py

```
def crop_and_flip(x: torch.Tensor, pad: int = 4) -> torch.Tensor:
    B, C, H, W = x.shape
    padded = F.pad(x, (pad, pad, pad, pad), mode="reflect")
    dy = torch.randint(0, 2 * pad + 1, (B, 1), device=x.device)
    dx = torch.randint(0, 2 * pad + 1, (B, 1), device=x.device)
    rows = (dy + torch.arange(H, device=x.device))[:, None, :, None]
    cols = (dx + torch.arange(W, device=x.device))[:, None, None, :]
    batch = torch.arange(B, device=x.device)[:, None, None, None]
    channel = torch.arange(C, device=x.device)[None, :, None, None]
    out = padded[batch, channel, rows, cols]
    flip = torch.rand(B, 1, 1, 1, device=x.device) < 0.5
    return torch.where(flip, out.flip(3), out)
```

Training uses SGD with Nesterov momentum, weight decay, a learning rate that rises linearly for the first quarter of training and then falls linearly to zero, and *label smoothing*. Label smoothing trains towards a target that puts 0.82 on the true class and 0.02 on each other class, instead of all of the probability on the true class, so the network gains nothing by pushing its logits towards infinity.

Sixteen epochs on CIFAR-10 — code/u1/cifar.py

```
model = ResNet9().to(dev).to(memory_format=torch.channels_last)
opt = torch.optim.SGD(
    model.parameters(), lr=args.lr, momentum=0.9, nesterov=True, weight_decay=5e-4
)
steps = args.epochs * (len(x) // args.batch)
warm = max(1, steps // 4)
schedule = torch.optim.lr_scheduler.LambdaLR(
    opt, lambda s: (s + 1) / warm if s < warm else max(0.0, (steps - s) / (steps - warm))
)
start = time.perf_counter()
for epoch in range(1, args.epochs + 1):
    order = torch.randperm(len(x), device=dev)
    for idx in order[: len(x) // args.batch * args.batch].view(-1, args.batch):
        loss = F.cross_entropy(model(crop_and_flip(x[idx])), y[idx], label_smoothing=0.2)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()
        schedule.step()
    test = accuracy(model, d["x_test"], d["y_test"])
    print(
        f"epoch {epoch:2d}  loss {loss.item():.3f}  test accuracy {test:.2%}  "
        f"{time.perf_counter() - start:5.0f} s",
        flush=True,
    )
```

ResNet9 on CIFAR-10

```
$ uv run -m u1.cifar
epoch  1  loss 1.591  test accuracy 55.71%     85 s
epoch  2  loss 1.391  test accuracy 64.59%    168 s
epoch  3  loss 1.321  test accuracy 69.14%    251 s
epoch  4  loss 1.276  test accuracy 73.71%    331 s
epoch  5  loss 1.236  test accuracy 75.47%    411 s
epoch  6  loss 1.155  test accuracy 72.97%    491 s
epoch  7  loss 1.145  test accuracy 84.20%    571 s
epoch  8  loss 1.103  test accuracy 85.87%    651 s
epoch  9  loss 1.083  test accuracy 86.32%    731 s
epoch 10  loss 1.057  test accuracy 87.00%    811 s
epoch 11  loss 1.041  test accuracy 88.99%    891 s
epoch 12  loss 1.028  test accuracy 89.13%    970 s
epoch 13  loss 0.997  test accuracy 89.24%   1050 s
epoch 14  loss 0.996  test accuracy 90.28%   1130 s
epoch 15  loss 0.993  test accuracy 91.36%   1210 s
epoch 16  loss 0.957  test accuracy 92.47%   1290 s
6.6M parameters; 16 epochs in 1290 s
nearest neighbour in feature space has the same class for 89.6% of 2,000 test images
```

The network reaches 92.5% in sixteen epochs, about twenty-two minutes on the M2 Pro. Accuracy climbs unevenly while the learning rate is high, as at epoch 6, and steadily as it falls. The training loss ends near 0.96 and cannot go much lower, because with label smoothing of 0.2 the least any prediction can score is the entropy of the smoothed target, about 0.87 nats. David Page's version of the recipe, with more refinements and 16-bit arithmetic, reaches 94% in under a minute on a single data-centre GPU.

The feature space

Without the classifier, the network maps each image to a vector of 512 features. Once the vectors are normalised to unit length, the cosine similarity between two of them measures how alike the network considers two images, and the most similar images to a query are its *nearest neighbours*:

Five nearest neighbours of six test images, by cosine similarity — code/u1/cifar.py

```
feats = F.normalize(embed(model.features, d["x_test"]), dim=1)
queries = [3, 10, 25, 42, 77, 101]
similar = feats[queries] @ feats.T
similar[torch.arange(len(queries)), queries] = -1
neighbours = similar.topk(5, dim=1).indices.cpu().numpy()
```

Twenty-nine of the thirty neighbours share their query's class, and over 2,000 test images the nearest neighbour has the same class 89.6% of the time, although the network was trained only to name images and not to place similar ones together. The neighbours share more than the class. An aeroplane seen against the sky finds others against the sky, and a bird among leaves finds birds among leaves. The one mistake is a fluffy brown dog whose neighbour is a long-haired cat, a confusion between dogs and cats that is common on CIFAR-10.



Figure 1.8.2. Six test images, left, and the five test images whose features are most similar to each. The network was trained only to name ten classes, yet its neighbours share pose, colour and setting as well as class.

Searching by similarity of learned vectors is the core of retrieval, and Chapter 3.7 does it with text.

Learning without labels

Supervised features need labels, and labels are expensive. *Contrastive learning* learns features from unlabelled images with a task that the data supplies itself: make two random views of each image, and train the network to recognise which views came from the same image. The views are much stronger distortions than the ones used for classification: a random crop of between a fifth and all of the image, resized; a random flip; random changes of brightness, contrast and saturation; and sometimes a conversion to grey.

A random view: crop and resize, flip, colour changes, grey — code/u1/vision.py

```
def random_view(x: torch.Tensor, min_scale: float = 0.2) -> torch.Tensor:
    B = x.shape[0]
    dev = x.device
    area = torch.empty(B, device=dev).uniform_(min_scale, 1.0)
    ratio = torch.exp(torch.empty(B, device=dev).uniform_(math.log(3 / 4), math.log(4 / 3)))
    sx = torch.sqrt(area * ratio).clamp(max=1.0)
    sy = torch.sqrt(area / ratio).clamp(max=1.0)
    tx = (torch.rand(B, device=dev) * 2 - 1) * (1 - sx)
    ty = (torch.rand(B, device=dev) * 2 - 1) * (1 - sy)
    flip = torch.where(torch.rand(B, device=dev) < 0.5, -1.0, 1.0)
    theta = torch.zeros(B, 2, 3, device=dev)
    theta[:, 0, 0], theta[:, 0, 2] = sx * flip, tx
    theta[:, 1, 1], theta[:, 1, 2] = sy, ty
    grid = F.affine_grid(theta, list(x.shape), align_corners=False)
    out = F.grid_sample(
        x * STD.to(dev) + MEAN.to(dev), grid, align_corners=False, padding_mode="reflection"
    )
    gray = (out * torch.tensor([0.299, 0.587, 0.114], device=dev).view(1, 3, 1, 1)).sum(
        1, keepdim=True
    )
    jitter = torch.rand(B, 1, 1, 1, device=dev) < 0.8
    brightness = torch.empty(B, 1, 1, 1, device=dev).uniform_(0.6, 1.4)
    contrast = torch.empty(B, 1, 1, 1, device=dev).uniform_(0.6, 1.4)
    saturation = torch.empty(B, 1, 1, 1, device=dev).uniform_(0.6, 1.4)
    mean = out.mean(dim=(1, 2, 3), keepdim=True)
    changed = (out * brightness - mean) * contrast + mean
    changed = gray + (changed - gray) * saturation
    out = torch.where(jitter, changed, out)
    to_gray = torch.rand(B, 1, 1, 1, device=dev) < 0.2
    out = torch.where(to_gray, gray.expand_as(out), out).clamp(0, 1)
    return (out - MEAN.to(dev)) / STD.to(dev)
```

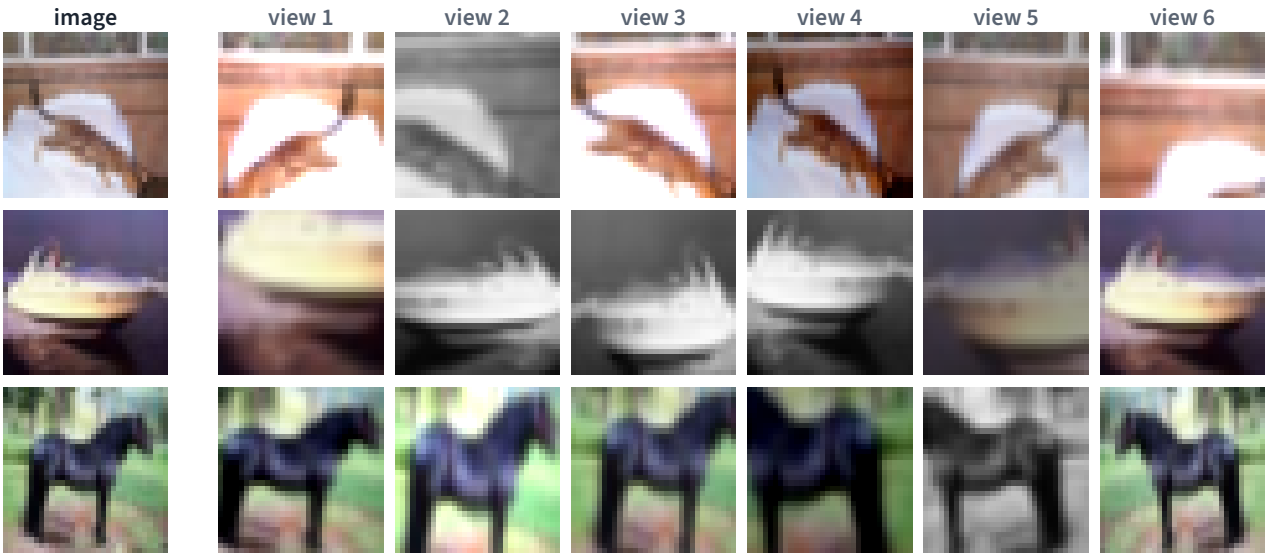


Figure 1.8.3. Three training images, left, and six random views of each. Recognising that two views come from the same image requires features that ignore position, scale and colour, and keep shape and texture.

The encoder, ResNet9 without its classifier, maps every view to 512 features, and a small *projection head* maps those to 128 dimensions where the loss is computed. For a batch of N images there are $2N$ views. Each view's *positive* is the other view of its image; its *negatives* are the remaining $2N-2$. The *InfoNCE* loss, from Aaron van den Oord and colleagues, treats this as a classification among the other $2N-1$ views. With z_i the normalised projection of view i, j its positive and τ a temperature,

$$\mathscr{L}_i = -\log \frac{\exp(z_i \cdot z_j/\tau)}{\sum_{k \neq i} \exp(z_i \cdot z_k/\tau)},$$

averaged over all $2N$ views. It is Chapter 1.3's cross-entropy with similarities for logits:

The InfoNCE loss over a batch of paired views — code/u1/contrastive.py

```
def info_nce(z: torch.Tensor, temperature: float) → torch.Tensor:
    z = F.normalize(z, dim=1)
    n = z.shape[0] // 2
    logits = z @ z.T / temperature
    logits.fill_diagonal_(float("-inf"))
    targets = torch.cat([torch.arange(n, 2 * n), torch.arange(0, n)]).to(z.device)
    return F.cross_entropy(logits, targets)
```

DERIVATION

Write p_{ik} for the softmax of $z_i \cdot z_k/\tau$ over $k \neq i$. By Chapter 1.3's result, the gradient of $\mathscr{L}_i$ with respect to the similarity $z_i \cdot z_k$ is $(p_{ik} - [k=j])/\tau$, and so, treating z_i as free,

$$\frac{\partial \mathscr{L}_i}{\partial z_i} = \frac{1}{\tau}\left(\sum_{k \neq i} p_{ik} z_k - z_j\right).$$

Descent moves z_i towards its positive and away from a weighted average of all candidates, weighted by how strongly the model currently confuses each with the positive. The most similar negatives, the hardest, receive most of the push, and a smaller temperature concentrates it further on them. An encoder that maps every image to the same vector makes all similarities equal and the loss $\log(2N-1)$, its value at chance. The negatives make that collapse costly.

Training is the usual loop, with no labels anywhere:

Contrastive training: two views of each image, one loss — code/u1/contrastive.py

```
encoder = ResNet9().to(dev).features
projector = nn.Sequential(nn.Linear(512, 512), nn.ReLU(), nn.Linear(512, 128)).to(dev)
params = list(encoder.parameters()) + list(projector.parameters())
opt = torch.optim.AdamW(params, lr=2e-3, weight_decay=1e-5)
start = time.perf_counter()
for epoch in range(1, args.epochs + 1):
    for idx in torch.randperm(len(x), device=dev)[
        : len(x) // args.batch * args.batch
    ].view(-1, args.batch):
        views = torch.cat([random_view(x[idx]), random_view(x[idx])])
        loss = info_nce(projector(encoder(views)), args.temperature)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        opt.step()
    print(
        f"epoch {epoch:2d}  InfoNCE loss {loss.item():.3f}  "
        f"{time.perf_counter() - start:5.0f} s",
        flush=True,
    )
```

Measuring a representation

A representation is good if a simple classifier can read the task off it. A *linear probe* freezes the encoder, computes features for the labelled training set, and trains only a linear classifier on them; its test accuracy measures how linearly separable the classes are in the representation. The program probes three encoders with identical architecture: an untrained one, the contrastive one, and the supervised network's.

A linear probe: standardised features, one linear layer, cross-entropy code/u1/vision.py

```python
def linear_probe(train_feats, y_train, test_feats, y_test, epochs: int = 100) -> float:
    mean, std = train_feats.mean(0), train_feats.std(0) + 1e-6
    a, b = (train_feats - mean) / std, (test_feats - mean) / std
    probe = nn.Linear(a.shape[1], 10).to(a.device)
    opt = torch.optim.AdamW(probe.parameters(), lr=1e-3, weight_decay=1e-4)
    for _ in range(epochs):
        for idx in torch.randperm(len(a), device=a.device).split(1000):
            loss = F.cross_entropy(probe(a[idx]), y_train[idx])
            opt.zero_grad()
            loss.backward()
            opt.step()
    with torch.no_grad():
        return (probe(b).argmax(1) == y_test).float().mean().item()
```

Linear probes of three encoders

```
$ uv run -m u1.contrastive
linear probe on an untrained encoder: 51.72%
epoch  1  InfoNCE loss 4.862     65 s
epoch  2  InfoNCE loss 4.228    128 s
epoch  3  InfoNCE loss 3.922    191 s
epoch  4  InfoNCE loss 3.772    254 s
epoch  5  InfoNCE loss 3.741    316 s
epoch  6  InfoNCE loss 3.565    379 s
epoch  7  InfoNCE loss 3.609    442 s
epoch  8  InfoNCE loss 3.489    505 s
epoch  9  InfoNCE loss 3.400    568 s
epoch 10  InfoNCE loss 3.469    631 s
epoch 11  InfoNCE loss 3.413    694 s
epoch 12  InfoNCE loss 3.385    756 s
epoch 13  InfoNCE loss 3.304    819 s
epoch 14  InfoNCE loss 3.270    882 s
epoch 15  InfoNCE loss 3.289    945 s
epoch 16  InfoNCE loss 3.223   1008 s
epoch 17  InfoNCE loss 3.219   1071 s
epoch 18  InfoNCE loss 3.148   1134 s
epoch 19  InfoNCE loss 3.166   1197 s
epoch 20  InfoNCE loss 3.214   1260 s
linear probe on the contrastive encoder: 75.20%
linear probe on Chapter 1.8's supervised encoder: 91.97%
```

A probe on the untrained network already reaches 52%, five times chance, because random convolutions followed by a maximum over positions still respond to colours, edges and textures, and those separate many images. Twenty epochs of contrastive learning, with no labels at all, raise it to 75%, more than half of the way to the 92% of the supervised network, which was trained with labels for the task the probe measures. The InfoNCE loss, 6.9 nats at chance, has fallen to about 3.2 when training stops, and is flat over the last few epochs. Contrastive learning needs far longer runs and larger batches than supervised learning. SimCLR's authors trained for hundreds of epochs with batches of 4,096 images, and their largest networks' probes matched a supervised network's on ImageNet.

> **KEY IDEA**
> Much of a trained network's usefulness lies in its representation, the features its layers compute. Convolutions build in locality and translation equivariance, and augmentation builds in invariances the task respects. A contrastive loss can learn useful features with no labels at all, by training the network to recognise which views of the data belong together.

> **PITFALL**
> Batch normalisation computes statistics over a batch. If an image's two views are normalised together, apart from most of their negatives, a network can learn to match views by the statistics they share instead of by what they show. On one device, as here, every view shares one set of

statistics and nothing leaks. Training across several GPUs, each normalising its own share of the batch, must synchronise the statistics or shuffle samples between devices to prevent it.

IN PYTORCH

`torch.nn.Conv2d`, `MaxPool2d` and `AdaptiveMaxPool2d` are this chapter's layers. `memory_format=torch.channels_last` stores images as height, width, channels, the layout convolution kernels on GPUs prefer. `torchvision.transforms.v2` provides the augmentations ready-made, and `torchvision.models` and the `timm` library provide trained networks whose features are a common starting point for new tasks.

Reading

- Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner, "Gradient-Based Learning Applied to Document Recognition", *Proceedings of the IEEE* 86(11), 1998.
- David Page, "How to Train Your ResNet", Myrtle.ai, 2018: the ResNet9 design and fast training on CIFAR-10.
- Aaron van den Oord, Yazhe Li and Oriol Vinyals, "Representation Learning with Contrastive Predictive Coding", 2018: the InfoNCE loss.
- Ting Chen, Simon Kornblith, Mohammad Norouzi and Geoffrey Hinton, "A Simple Framework for Contrastive Learning of Visual Representations", ICML 2020.

Labs

LAB 1.8.1 The effect of augmentation — 1.5 hours

Goal. Measure augmentation as regularisation.

Code. `code/u1/cifar.py`

1. Train ResNet9 three times: without augmentation, with crops only, and with crops and flips.
2. Record training and test accuracy at the end of each run.

DONE WHEN

- [ ] Your table has three rows with both accuracies.
- [ ] You can explain the change in the gap between them using Chapter 1.6's ideas.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

LAB 1.8.2 The fastest run to 93% — 2 hours

Goal. Find the cheapest training that reaches a target.

Code. `code/u1/cifar.py`

1. Find the shortest training run on your machine that reaches 93% test accuracy, varying the number of epochs, the batch size, the learning rate and the widths.
2. Record every run's settings, time and accuracy.

DONE WHEN

- [ ] You have a run that reaches 93%, with its time.
- [ ] Your log shows which change saved the most time.

EVIDENCE FOR THE NOTEBOOK
- The log of runs.

LAB 1.8.3 Which views matter — 2 hours

Goal. Find which augmentations contrastive learning depends on.

Code. `code/u1/contrastive.py`

1. Train the contrastive encoder for 10 epochs three times: with crops and flips only, with crops, flips and colour changes, and with every augmentation.
2. Probe each.

DONE WHEN
- [ ] You have three probe accuracies.
- [ ] You can explain, in terms of what the task can be solved by, why removing colour changes hurts.

EVIDENCE FOR THE NOTEBOOK
- The accuracies and the explanation.

LAB 1.8.4 Few labels — 1.5 hours

Goal. Compare contrastive features with supervised training when labels are scarce.

Code. `code/u1/contrastive.py`

1. Keep 50 labelled images per class. Train ResNet9 on those 500 images alone, and train a linear probe on the contrastive features of the same 500.
2. Compare both test accuracies with the probe trained on all 50,000 labels.

DONE WHEN
- [ ] Your table has three accuracies.
- [ ] You can state how many labels the contrastive features are worth, roughly, and how you would measure that more precisely.

EVIDENCE FOR THE NOTEBOOK
- The table.

Problem set

1. How many weights does the first convolution of ResNet9 have? How many would a dense layer from a $32 \times 32 \times 3$ image to a $32 \times 32 \times 64$ feature map have?
2. Follow the longest path through ResNet9 and compute the receptive field of a unit at the input to the final pooling layer. Does every unit see the whole image?
3. The network's features are a maximum over all positions. Show that they are unchanged when the input is shifted by a multiple of eight pixels, ignoring the image's edges. Why not by one pixel?
4. With a batch of 512 images, what is the InfoNCE loss of an encoder that maps every image to the same vector? Why can a projection to 128 dimensions not make every negative similarity equal to -1?
5. A linear probe on an untrained encoder does far better than chance. What in a random convolutional network could make the classes partly separable?

COMPLETION CHECKLIST

- [ ] I can explain what convolutions build into a network and count their weights.
- [ ] I can train a convolutional network with augmentation and explain what each part of the recipe does.
- [ ] I can find nearest neighbours in a feature space and interpret them.
- [ ] I can derive the InfoNCE loss's gradient and train an encoder contrastively.
- [ ] I can compare representations with a linear probe.
- [ ] Lab 1.8.1–1.8.4 done-when criteria all hold, with evidence filed.

UNIT 1

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who trains models for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
1.1	Tensors and what they cost	☐	☐	
1.2	Gradients by machine	☐	☐	
1.3	Models as probability	☐	☐	
1.4	Optimisers	☐	☐	
1.5	Neural networks	☐	☐	
1.6	Generalisation	☐	☐	
1.7	Training deep networks	☐	☐	
1.8	Representations	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ Derive, on paper, the backward steps of matrix multiplication, broadcasting and log-softmax, and explain how `tiny` implements each.
- ☐ Derive the mean squared error and the cross-entropy from maximum likelihood, and say what each assumes about the data.
- ☐ Explain, with your Chapter 1.4 measurements, when momentum helps, when Adam helps, and what a learning-rate schedule is for.
- ☐ Explain why a 30-layer network fails to train with PyTorch's default initialisation, and what each fix in Chapter 1.7 changes.
- ☐ Describe how you would decide whether a model is overfitting, and four ways to reduce it, with the evidence from your Chapter 1.6 runs.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____